

Vivekanand Education Society's Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai
Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.



Department of Information Technology

CERTIFICATE

This is to certify that **Hitesh Rohra** of **D15A** semester **VI**, have successfully completed necessary experiments in the **MAD & PWA Lab** under my supervision in **VES Institute of Technology** during the academic year **2024-2025**.

Lab Assistant

Subject Teacher

Mrs. Kajal Joseph

Principal

Head of Department

Dr. Mrs. Shalu Chopra

Name of the Course : MAD & PWA Lab

Course Code : ITL604

Year/Sem/Class : D15A

A.Y.: 24-25

Faculty Incharge : Mrs. Kajal Joseph.

Lab Teachers : Mrs. Kajal Joseph.

Email : kajal.jewani@ves.ac.in

Programme Outcomes: The graduate will be able to:

PO1) Basic Engineering knowledge: An ability to apply the fundamental knowledge in mathematics, science and engineering to solve problems in Computer engineering.

PO2) Problem Analysis: Identify, formulate, research literature and analyze computer engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences and computer engineering and sciences.

PO3) Design/ Development of Solutions: Design solutions for complex computer engineering problems and design system components or processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal and environmental considerations.

PO4) Conduct investigations of complex engineering problems using research-based knowledge and research methods including design of experiments, analysis and interpretation of data and synthesis of information to provide valid conclusions.

PO5) Modern Tool Usage: Create, select and apply appropriate techniques, resources and modern computer engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6) The Engineer and Society: Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to computer engineering practice.

PO7) Environment and Sustainability: Understand the impact of professional computer engineering solutions in societal and environmental contexts and demonstrate knowledge of and need for sustainable development.

PO8) Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of computer engineering practice.

PO9) Individual and Team Work: Function effectively as an individual, and as a member or leader in diverse teams and in multidisciplinary settings.

PO10) Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations and give and receive clear instructions.

Project Title: District (Booking App)**Roll No. 46**

PO11) Project Management and Finance: Demonstrate knowledge and understanding of computer engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12) Life-long Learning: Recognize the need for and have the preparation and ability to engage in independent and lifelong learning in the broadest context of technological change.

Program specific Outcomes

PSO1) An ability to manage and analyze data / information effectively for making better decisions.

PSO2) Demonstrate the ability to use state of the art technologies and tools including Free and Open Source Software (FOSS) tools in developing software.

Lab Objectives:

Sr. No.	Lab Objectives
The Lab experiments aims:	
1	Learn the basics of the Flutter framework.
2	Develop the App UI by incorporating widgets, layouts, gestures and animation
3	Create a production ready Flutter App by including files and firebase backend service.
4	Learn the Essential technologies, and Concepts of PWAs to get started as quickly and efficiently as possible
5	Develop responsive web applications by combining AJAX development techniques with the jQuery JavaScript library.
6	Understand how service workers operate and also learn to Test and Deploy PWA.

Lab Outcomes:

Sr. No.	Lab Outcomes	Cognitive levels of attainment as per Bloom's Taxonomy
On Completion of the course the learner/student should be able to:		
1	Understand cross platform mobile application development using Flutter framework	L1, L2
2	Design and Develop interactive Flutter App by using widgets, layouts, gestures and animation	L3
3	Analyze and Build production ready Flutter App by incorporating backend services and deploying on Android / iOS	L3, L4
4	Understand various PWA frameworks and their requirements	L1, L2
5	Design and Develop a responsive User Interface by applying PWA Design techniques	L3
6	Develop and Analyse PWA Features and deploy it over app hosting solutions	L3, L4

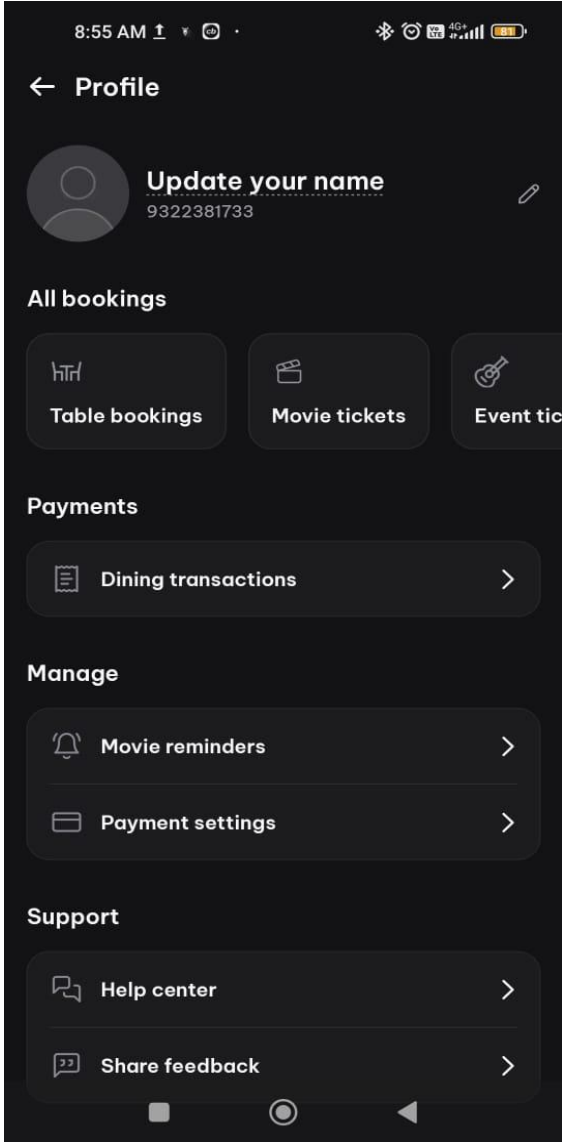
Index

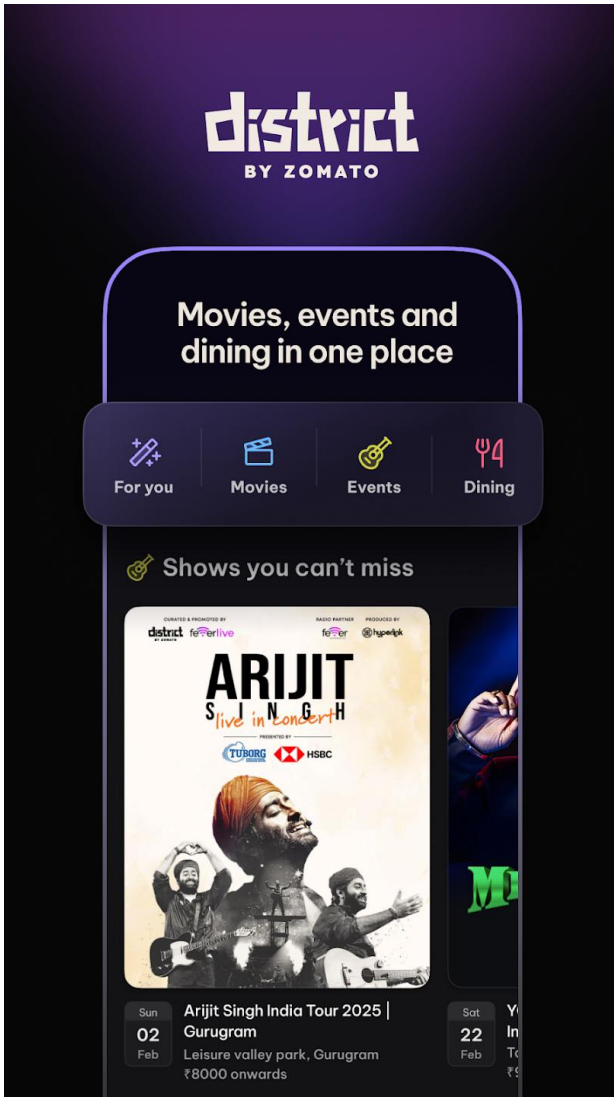
Sr. No	Experiment Title	LO	DOP	DOS	Grade
1.	To install and configure the Flutter Environment	LO1			
2.	To design Flutter UI by including common widgets.	LO2			
3.	To include icons, images, fonts in Flutter app	LO2			
4.	To create an interactive Form using form widget	LO2			
5.	To apply navigation, routing and gestures in Flutter App	LO2			
6.	To Connect Flutter UI with fireBase database	LO3			
7.	To write meta data of your Ecommerce PWA in a Web app manifest file to enable “add to homescreen feature”.	LO4			
8.	To code and register a service worker, and complete the install and activation process for a new service worker for the E-commerce PWA	LO5			
9.	To implement Service worker events like fetch, sync and push for E-commerce PWA	LO5			
10.	To study and implement deployment of Ecommerce PWA to GitHub Pages.	LO5			
11.	To use google Lighthouse PWA Analysis Tool to test the PWA functioning.	LO6			
12.	Assignment-1	LO1,LO2 ,LO3			
13.	Assignment-2	LO4,LO5 ,LO6			

MPL & PWA LAB PREREQUISITE

AIM: - Selecting features for application development, the features should comprise of:

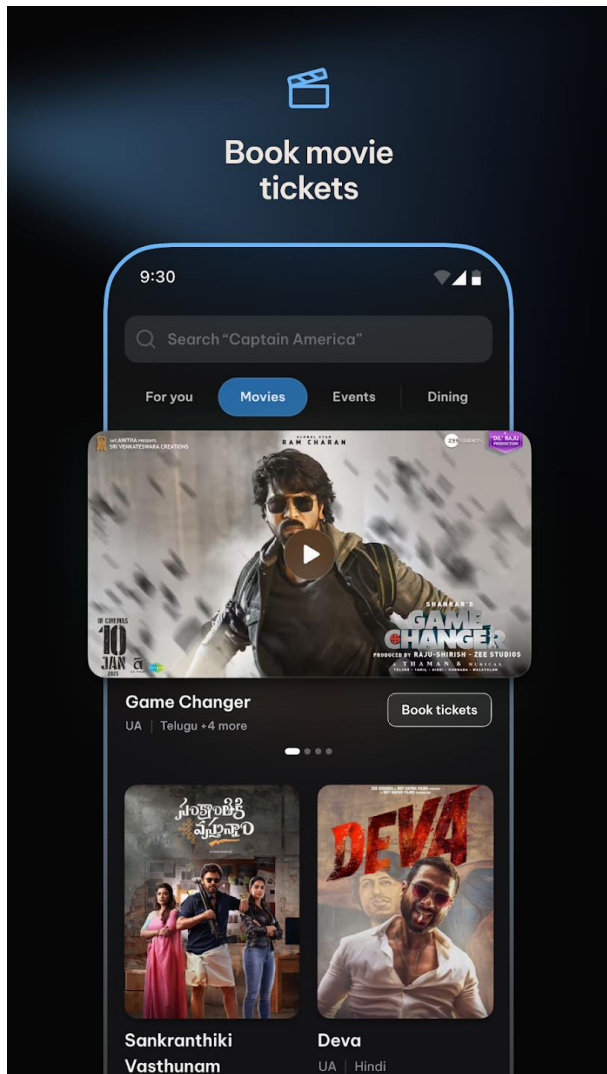
1. Common Widgets
2. Should include icons, images, charts etc.
3. Should have an interactive form.
4. Should apply navigation, routing and gestures
5. Should connect with FireBase database

Screen Shot	Features
	Profile Page • Purpose: Enables users to register or log in by Google Account, or optionally by Facebook login or another email ID. • Features: 1. “Continue with Google” Button 2. Option to use other email • Firestore Integration and Google Sign In Ensures secure login by storing user details in Firestore Authentication. GoogleSignInClient API can validate user identity based on Google Account credentials. • Facebook account sign in The LoginManager in Facebook SDK and handle sign in requests with Facebook account.



HomePage

1. **Navigation Buttons:** I'd use a BottomNavigationBar or a Drawer for easy navigation between different screens like Home, Profile, and Bookings.
2. **Icons for Notifications, Profile, and Search:** I'd place IconButton widgets in the app bar for quick access to notifications, the user's profile, and a search function. I would use icons like Icons.notifications, Icons.account_circle, and Icons.search to make it intuitive.
3. **Search Bar:** I'd add a TextField or use SearchDelegate to create a search bar where users can type and search for movies.
4. **Display Recommended Movies:** To show recommended movies, I'd use a ListView or GridView to dynamically display a list of movies. The data come from Firebase Firestore, so the recommendations would update as needed.



Movies Details Page:
Display all the information about the
Movie to user



Select your seats & share seat map



Game Changer
UA | Telugu | 2D
PVR Director's Cut, Ambience Mall, Gurga...

Fri
10 Jan

03:10 PM
DIRECTOR'S
3D

06:00 PM
DIRECTOR'S
3D

08:50 PM
DIRECTOR'S
3D

PLATINUM SUPERIOR: ₹1250

H	☐	☐	☐	☐	☐	☐	☐
G	☐	☐	☐	☐	☐	☐	☐
F	☐	☐	☐	☐	☐	☐	☐
E	☐	☐	☐	☒	☒	☐	☐

PLATINUM: ₹1050

D	☐	☐	☐	☐	☐	☐	☐
C	☐	☐	☐	☐	☐	☐	☐
B	☐	☐	☐	☐	☐	☐	☐
A	☐	☐	☒	☒	☐	☐	☐

SCREEN THIS WAY

☐ Available ☒ Occupied ☐ Selected

district
BY ZOMATO

#SeeYouThere

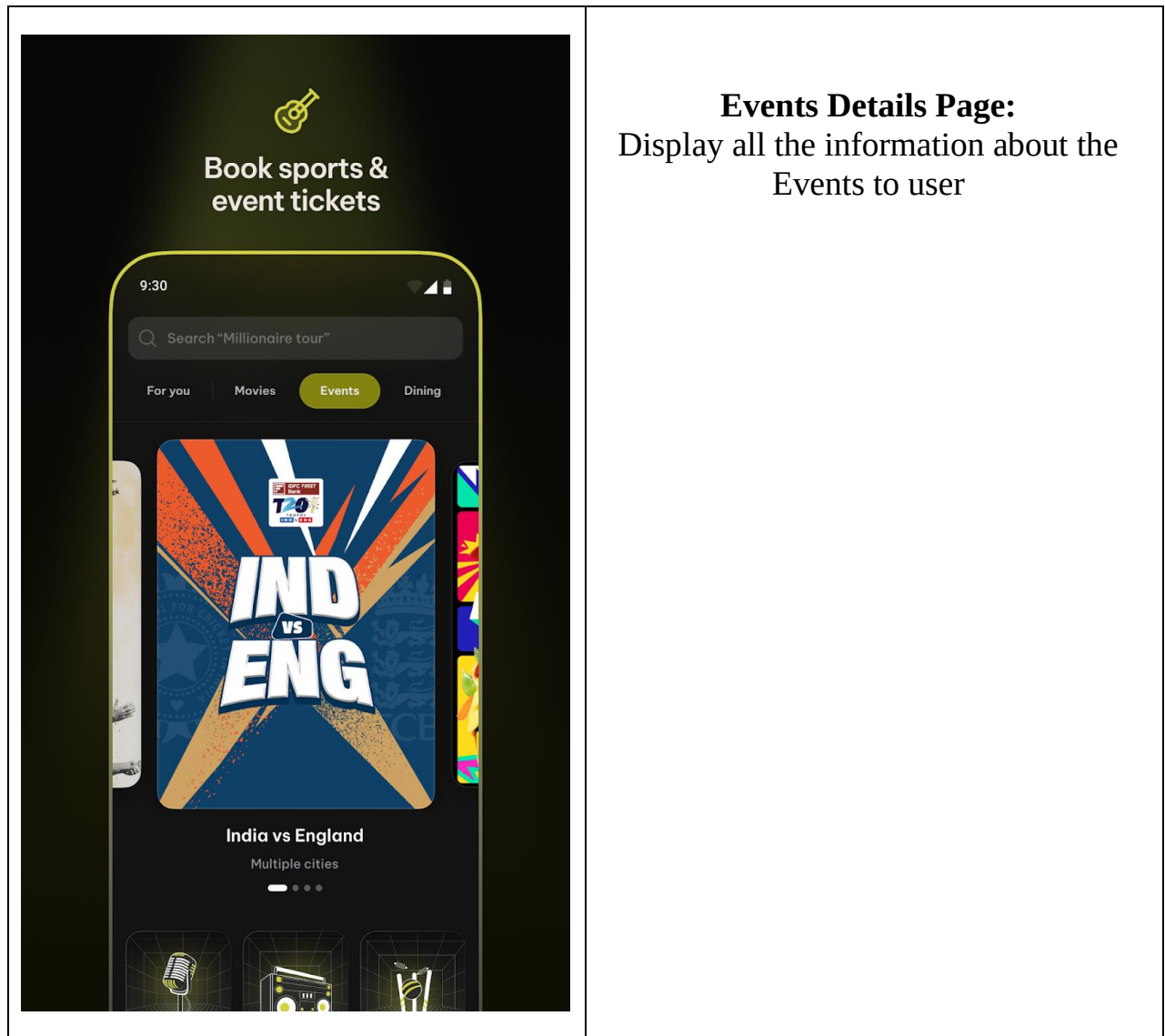
Share seat map

Show Booking Details

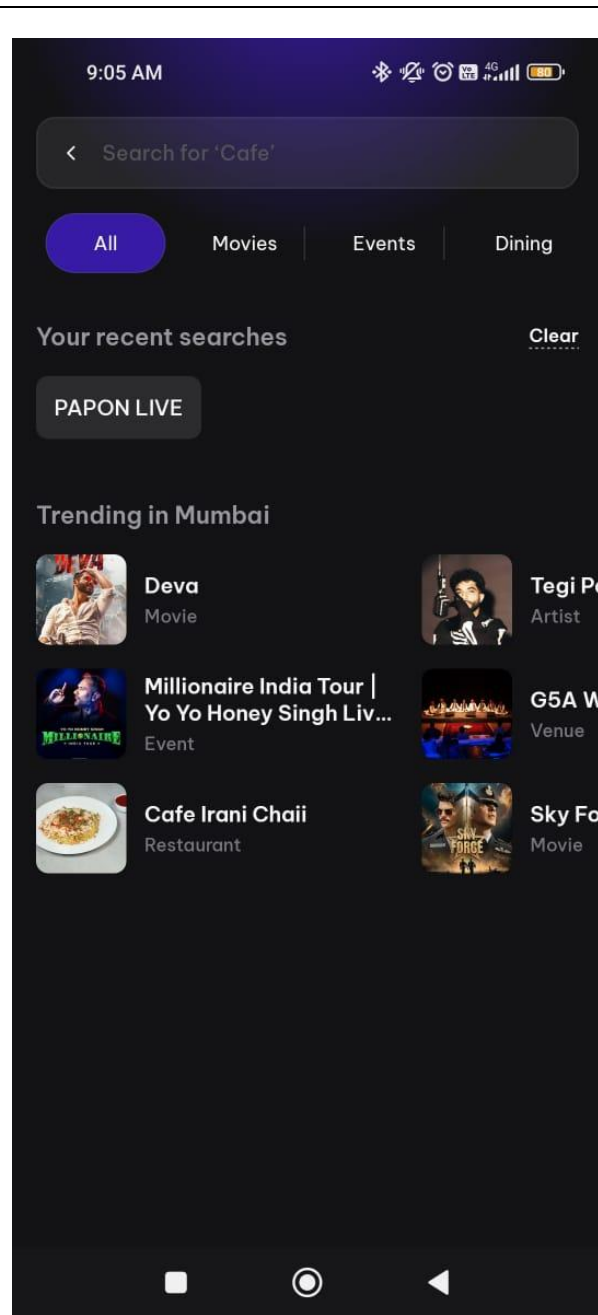
User will select no of seats .

Further Booking Details:

User will proceed with booking. After this page user will navigate to payment page.



Events Details Page:
Display all the information about the Events to user



Search Page

- **Purpose :**
To let the user search for movies, Events, Live Shows, Restaurants Dining, StandUp Shows, etc

MAD & PWA Lab Journal

Experiment No.	01
Experiment Title.	To install and configure the Flutter Environment
Roll No.	46
Name	Hitesh Rohra
Class	D15A
Subject	MAD & PWA Lab
Lab Outcome	LO1: Understand cross platform mobile application development using Flutter framework
Grade:	

EXPERIMENT NO: - 01

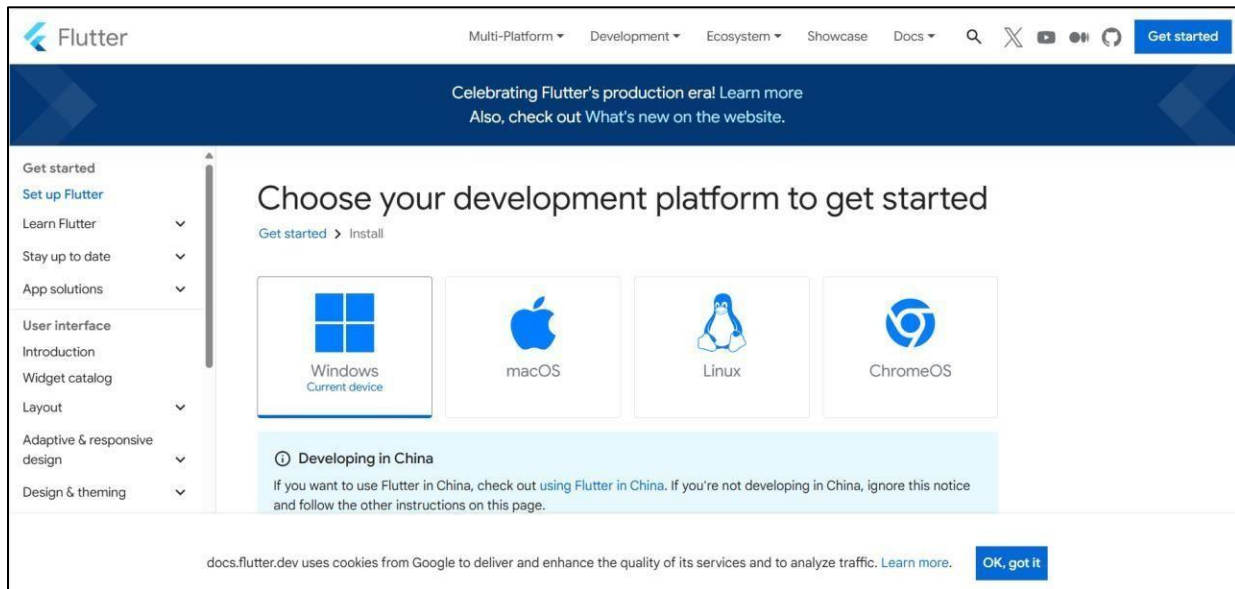
Name:- Hitesh Rohra

Class:- D15A

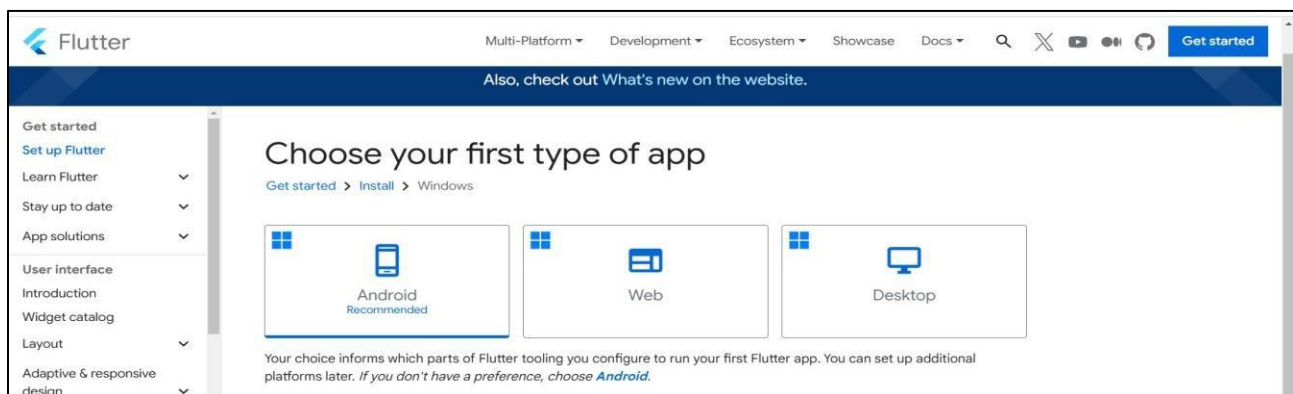
Roll:No:- 46

AIM:- Installation and Configuration of Flutter Environment.

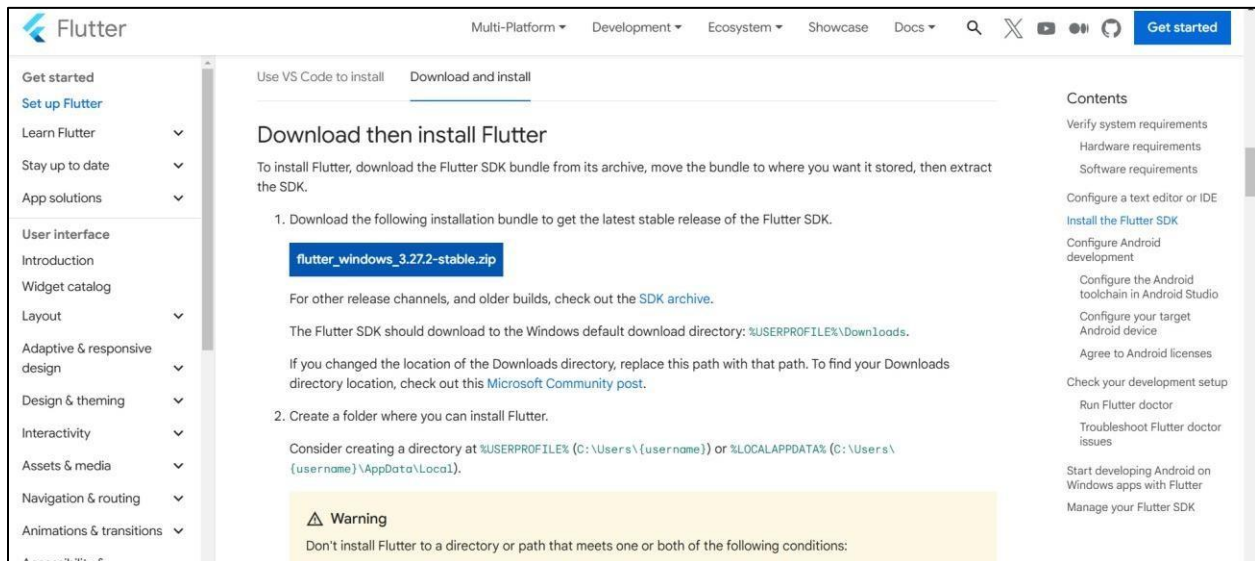
Step 1: Go to the official Flutter website: <https://docs.flutter.dev/get-started/install>



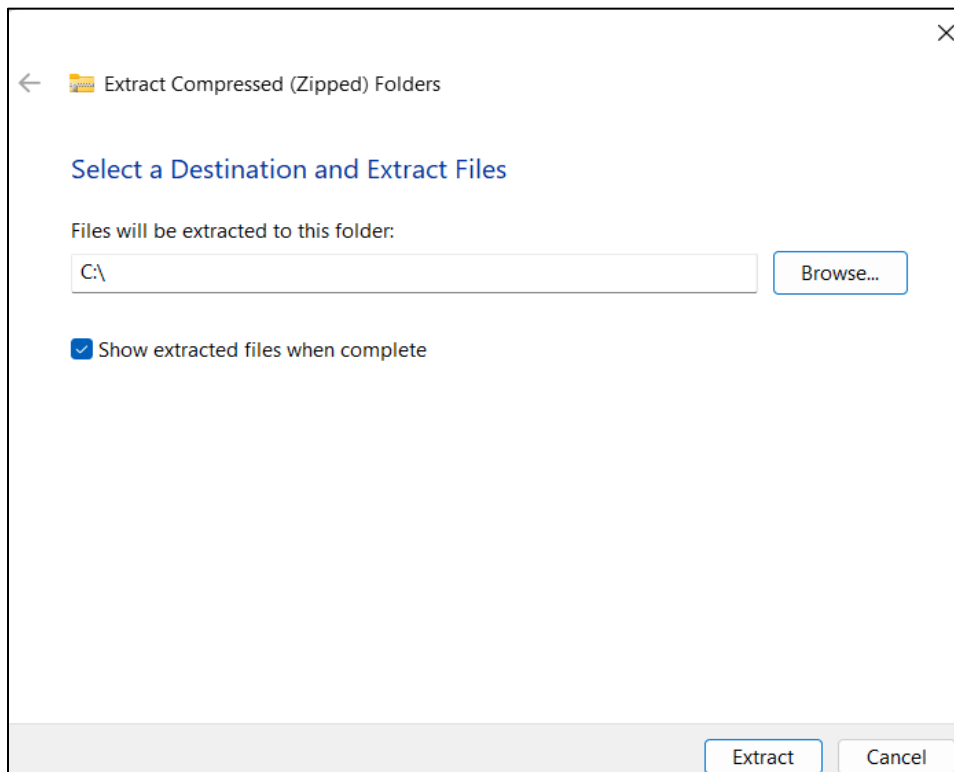
Step 2: To download the latest Flutter SDK, click on the Windows icon > Android



Step 3: For Windows, download the stable release (a .zip file).



Step 4: Extract the ZIP file to a folder (e.g., C:\flutter).

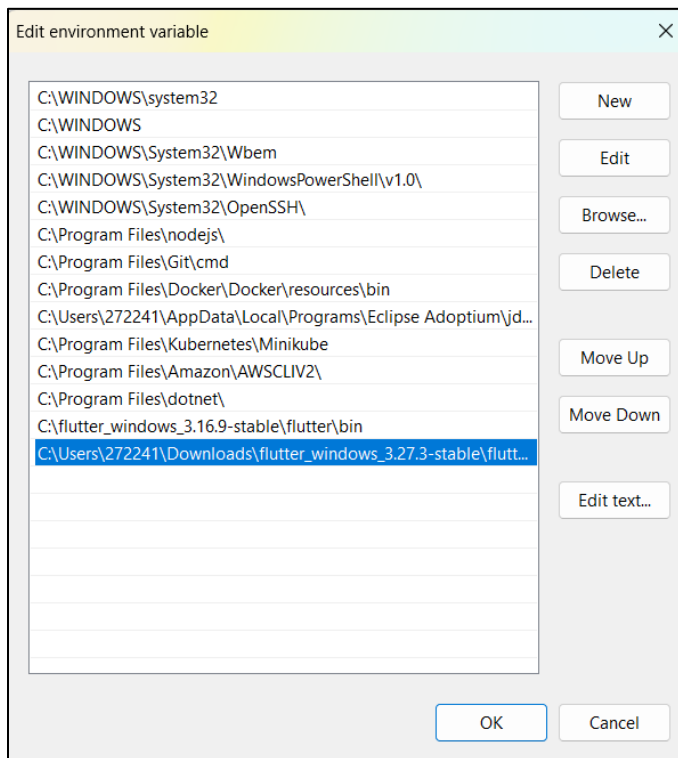


Step 5 :- Add Flutter to System PATH

Right-click on the Start Menu > System > Advanced system settings > Environment Variables.

Under System Variables, find Path and click Edit.

Add the full path to the flutter/bin directory (e.g., C:\flutter\bin).



Step 6 :- Now, run the \$ flutter command in command prompt.

```
Command Prompt - flutter
Microsoft Windows [Version 10.0.22631.4751]
(c) Microsoft Corporation. All rights reserved.

C:\Users\272241>flutter
Manage your Flutter app development.

Common commands:

  flutter create <output directory>
    Create a new Flutter project in the specified directory.

  flutter run [options]
    Run your Flutter application on an attached device or in an emulator.

Usage: flutter <command> [arguments]

Global options:
-h, --help                Print this usage information.
-v, --verbose              Noisy logging, including all shell commands executed
                           If used with "--help", shows hidden options. If used
                           with "--verbose", shows diagnostic information. (Use "-vv" to force verbose
                           logging.)
-d, --device-id            Target device id or name (prefixes allowed).
```


Step 7:- Run the \$ flutter doctor command. This command checks for all the requirements of Flutter app development and displays a report of the status of your Flutter installation

```
C:\Users\272241>flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.27.3, on Microsoft Windows [Version 10.0.22631.4751], locale en)
[✓] Windows Version (Installed version of Windows is version 10 or higher)
[✓] Android toolchain - develop for Android devices (Android SDK version 34.0.0)
[✓] Chrome - develop for the web
[!] Visual Studio - develop Windows apps (Visual Studio Community 2022 17.12.4)
    X Visual Studio is missing necessary components. Please re-run the Visual Studio installer
      development with C++ workload, and include these components:
        MSVC v142 - VS 2019 C++ x64/x86 build tools
          - If there are multiple build tool versions available, install the latest
        C++ CMake tools for Windows
        Windows 10 SDK
[✓] Android Studio (version 2023.1)
[✓] VS Code (version 1.96.4)
[✓] Connected device (3 available)
[✓] Network resources

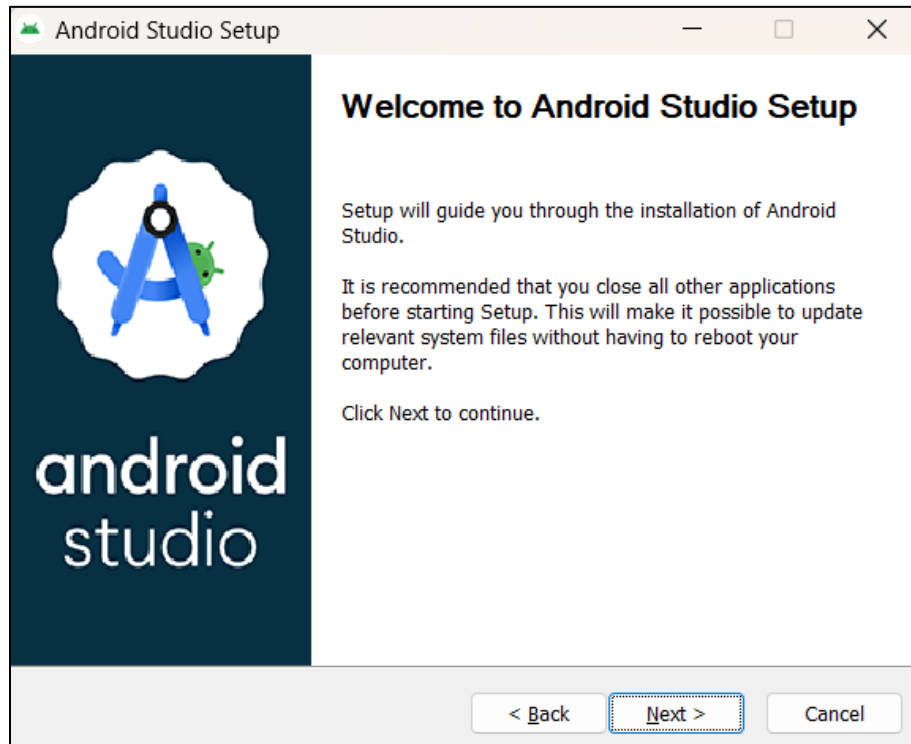
! Doctor found issues in 1 category.
```

Step 8 :- Go to Android Studio and download the installer.

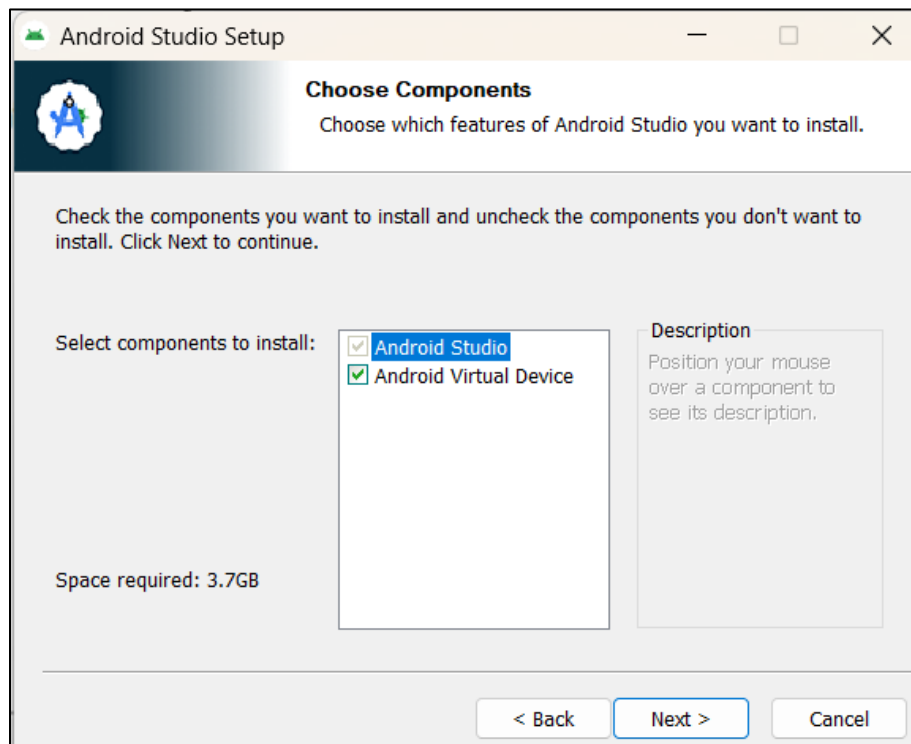
Download the latest version of Android Studio. For more information, see the [Android Studio release notes](#).

Platform	Android Studio package	Size	SHA-256 checksum
Windows (64-bit)	android-studio-2024.2.2.13-windows.exe Recommended	1.2 GB	7d93dd9bf3539f948f609b1968507b1f502bfe965d2d44bd38a17ff26cb5dd3e
Windows (64-bit)	android-studio-2024.2.2.13-windows.zip No .exe installer	1.2 GB	855945962ff9b84ea49ce39de0bf4189dbf451ae37a6fab7999da013b046b7f7
Mac (64-bit)	android-studio-2024.2.2.13-mac.dmg	1.3 GB	acfbbe54d6ce8cf21f19b43510c7addcb9dde2824282f205fd1331be77d2e613
Mac (64-bit, ARM)	android-studio-2024.2.2.13-mac_arm.dmg	1.3 GB	688f8d007e612f3f0c18f316179079dc4565f93d8d1e6a7dad80c4cfe35df7
Linux (64-bit)	android-studio-2024.2.2.13-linux.tar.gz	1.3 GB	b7fe1ed4a7959bdaca7a8fd57461dbbf9a205eb23cc218ed82ed88eb998cb5

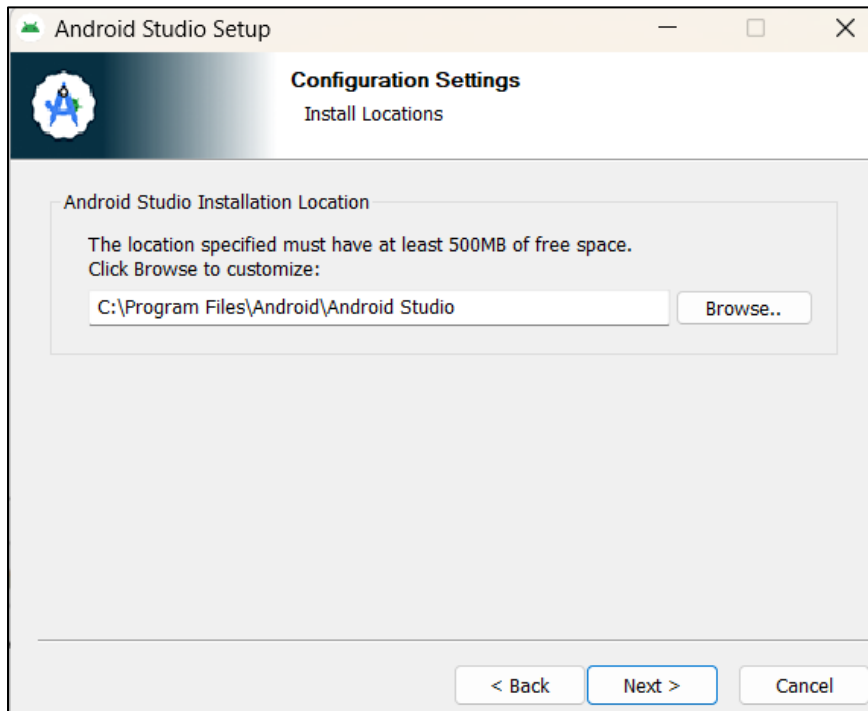
Step 8.1: - When the download is complete, open the .exe file and run it. You will get the following dialog box



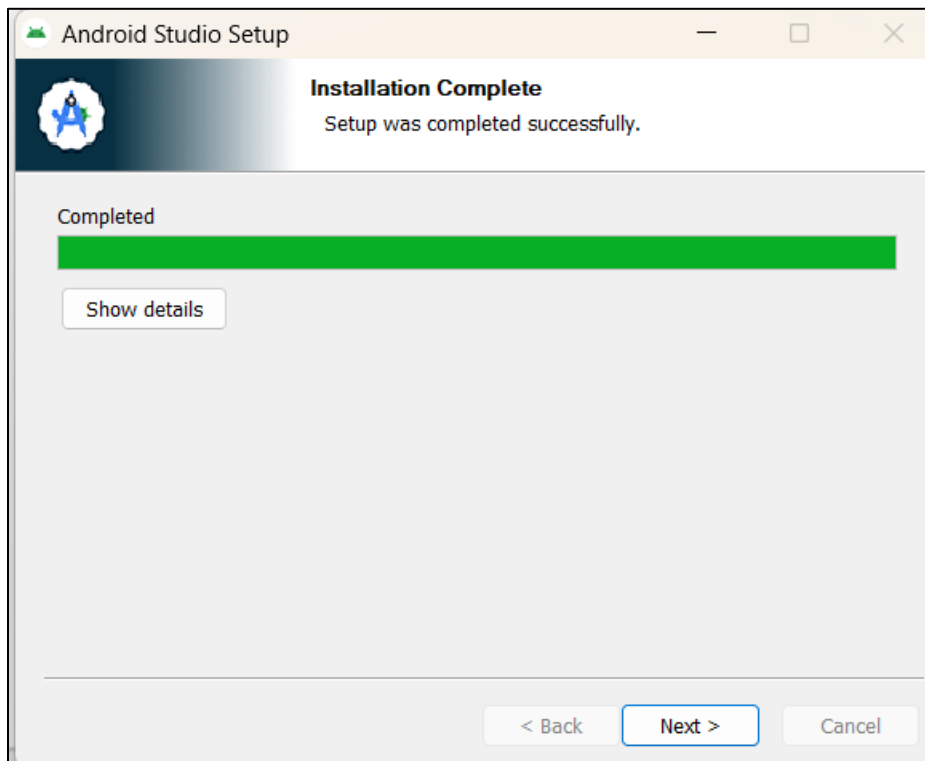
Step 8.2: - Select all the Checkboxes and Click on 'Next' Button.

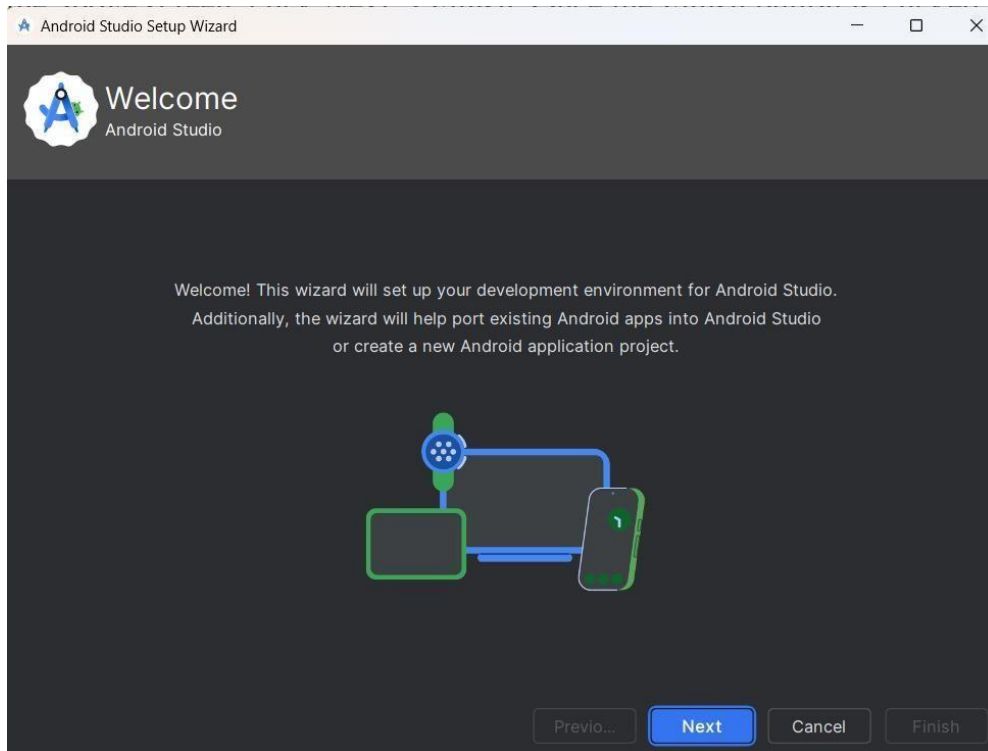


Step 8.3: - Change the destination as per your convenience and click on 'Next' Button.



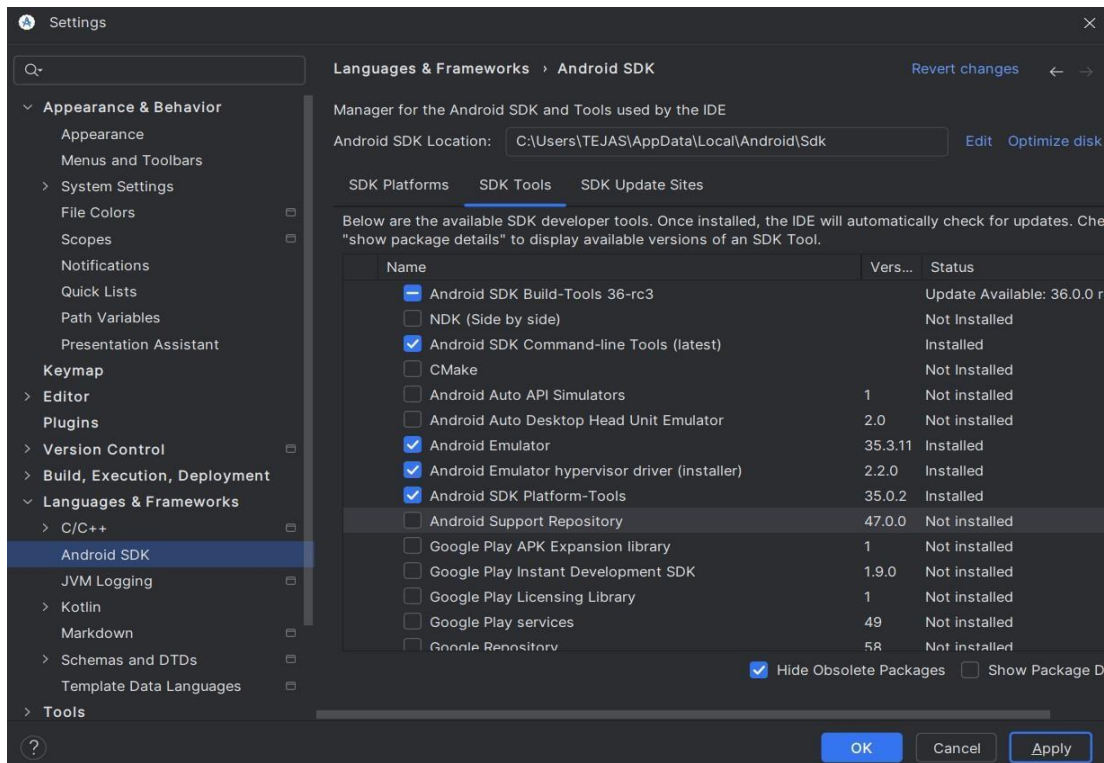
Step 8.4: - Follow the steps of the installation wizard. Once the installation wizard completes, you will get the following screen.





Step 8.5: - Go to Preferences > Appearance & Behavior > System Settings > Android SDK.

Select the SDK Tools tab and check Android SDK Command-line Tools and Install it.



Step 9: - Open a terminal and run the following command

```
C:\Users\272241>flutter doctor --android-licenses
```

Warning: Errors during XML parse:

```
Warning: Additionally, the fallback loader failed to parse the XML.
Warning: Errors during XML parse:      ] 64% Fetch remote repository...
Warning: Additionally, the fallback loader failed to parse the XML.ry...
[=====] 100% Computing updates...
All SDK package licenses accepted.
```

```
C:\Users\272241>
```

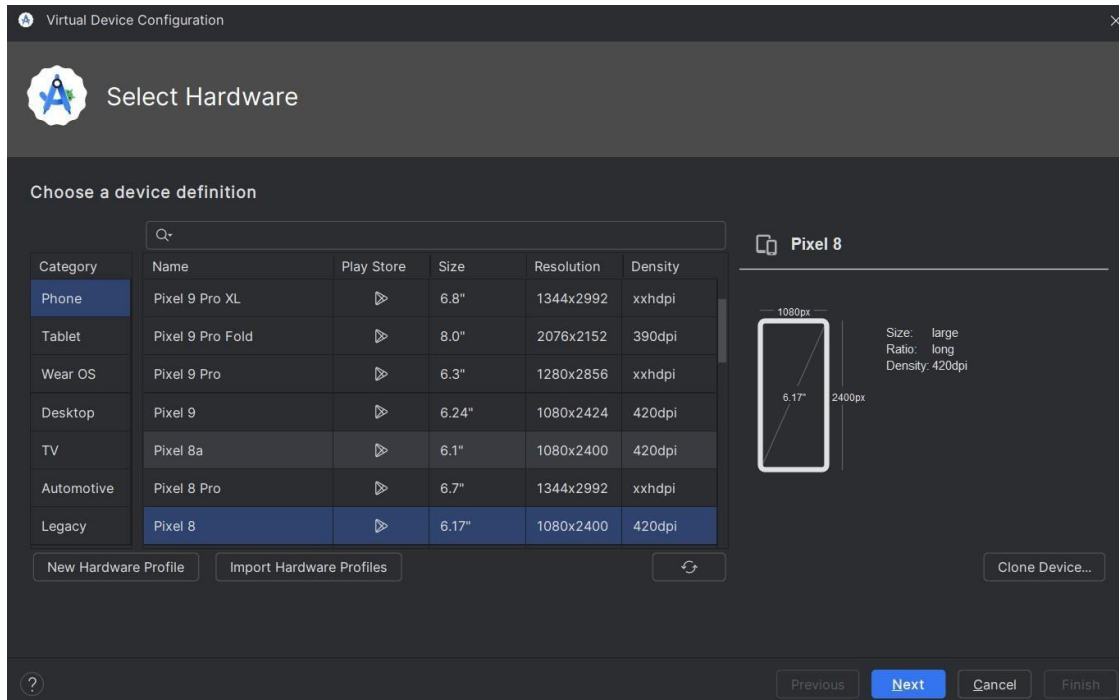
```
C:\Users\272241>flutter doctor
```

Doctor summary (to see all details, run flutter doctor -v):

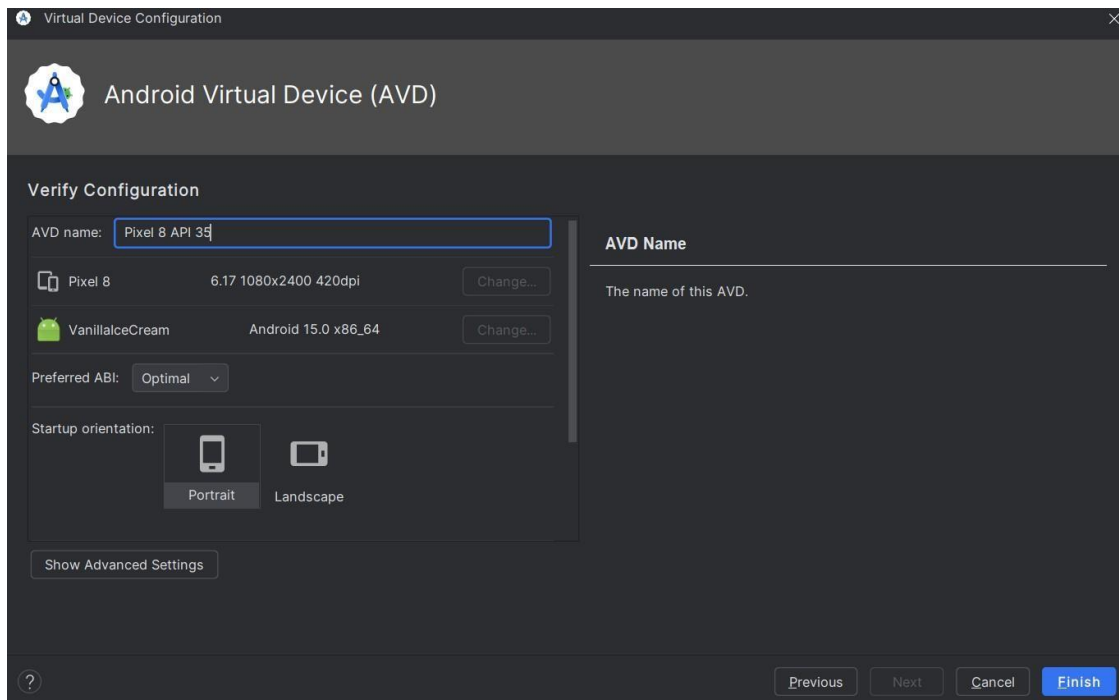
```
[✓] Flutter (Channel stable, 3.27.3, on Microsoft Windows [Version 10.0.22631.4751], locale en-US)
[✓] Windows Version (Installed version of Windows is version 10 or higher)
[✓] Android toolchain - develop for Android devices (Android SDK version 34.0.0)
[✓] Chrome - develop for the web
[!] Visual Studio - develop Windows apps (Visual Studio Community 2022 17.12.4)
    X Visual Studio is missing necessary components. Please re-run the Visual Studio installer
      development with C++ workload, and include these components:
        MSVC v142 - VS 2019 C++ x64/x86 build tools
        - If there are multiple build tool versions available, install the latest
        C++ CMake tools for Windows
        Windows 10 SDK
[✓] Android Studio (version 2023.1)
[✓] VS Code (version 1.96.4)
[✓] Connected device (3 available)
[✓] Network resources
```

! Doctor found issues in 1 category.

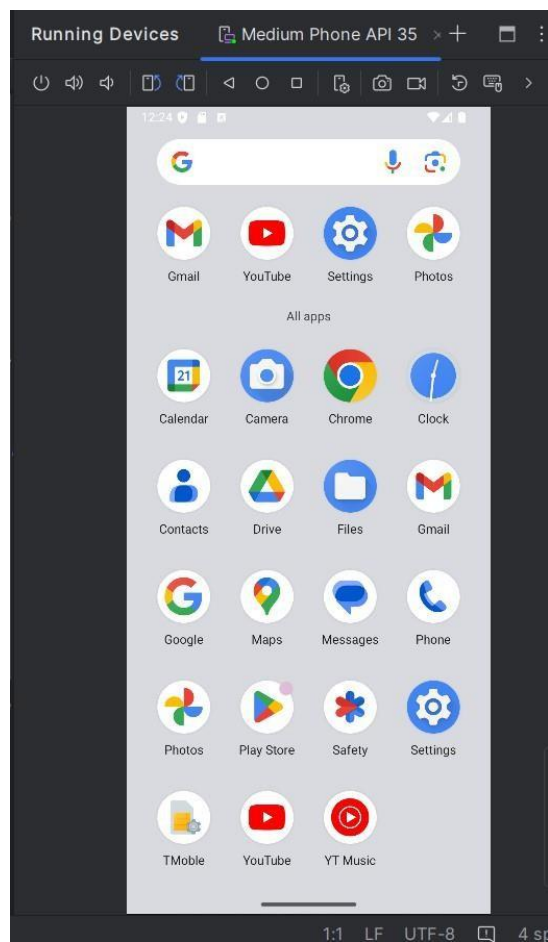
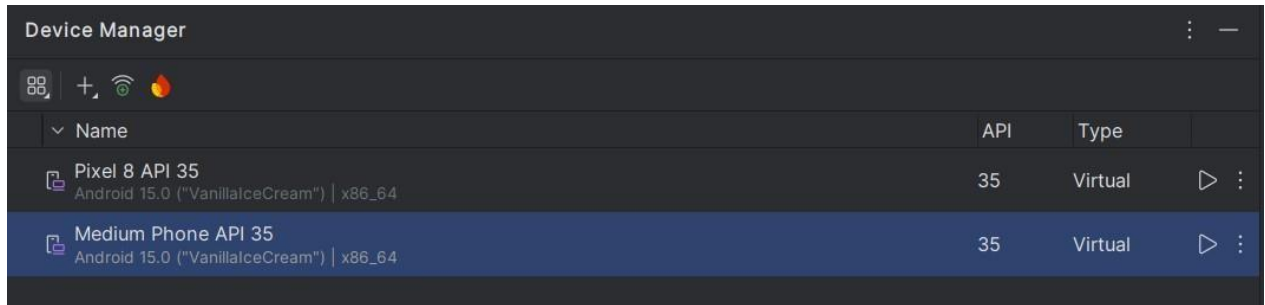
Step 10: - Next, you need to set up an Android emulator. It is responsible for running and testing the Flutter application



Step 10.1: - Open Android Studio and go to Tools > AVD Manager. Create a new virtual device.

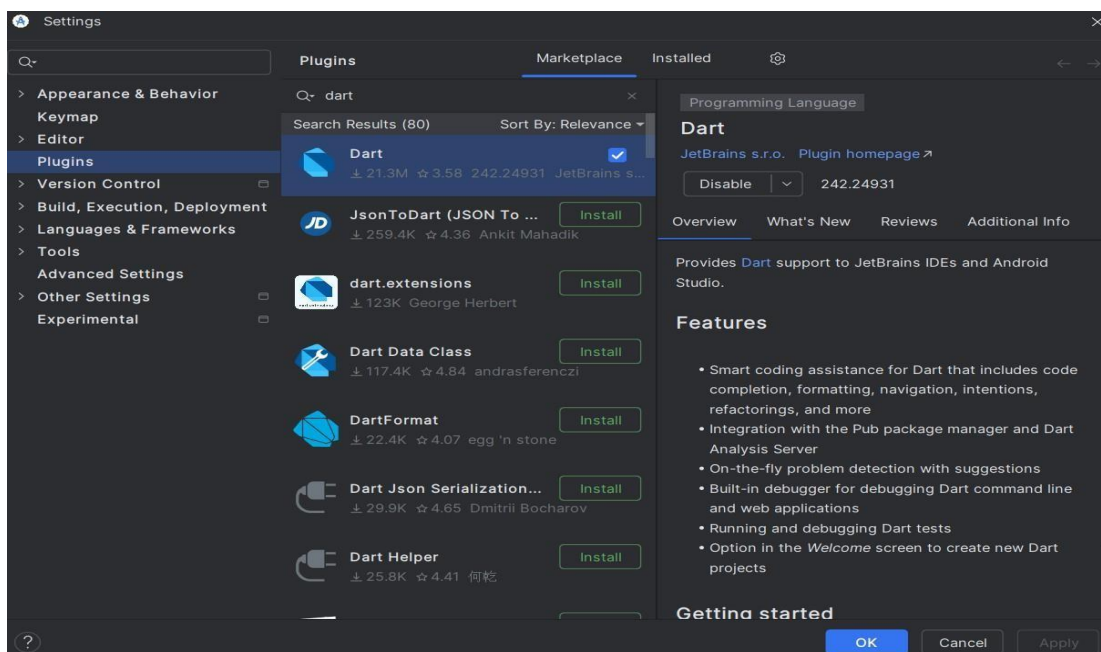
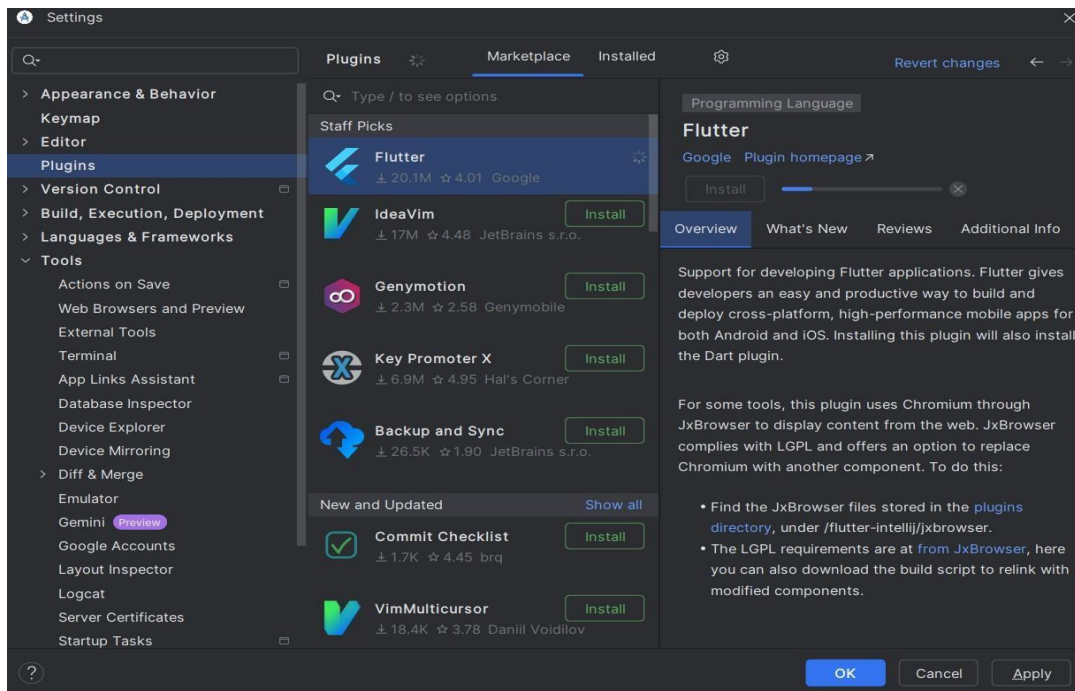


Step 10.2: - Click on the icon pointed into the red color rectangle. The Android emulator displayed as below screen



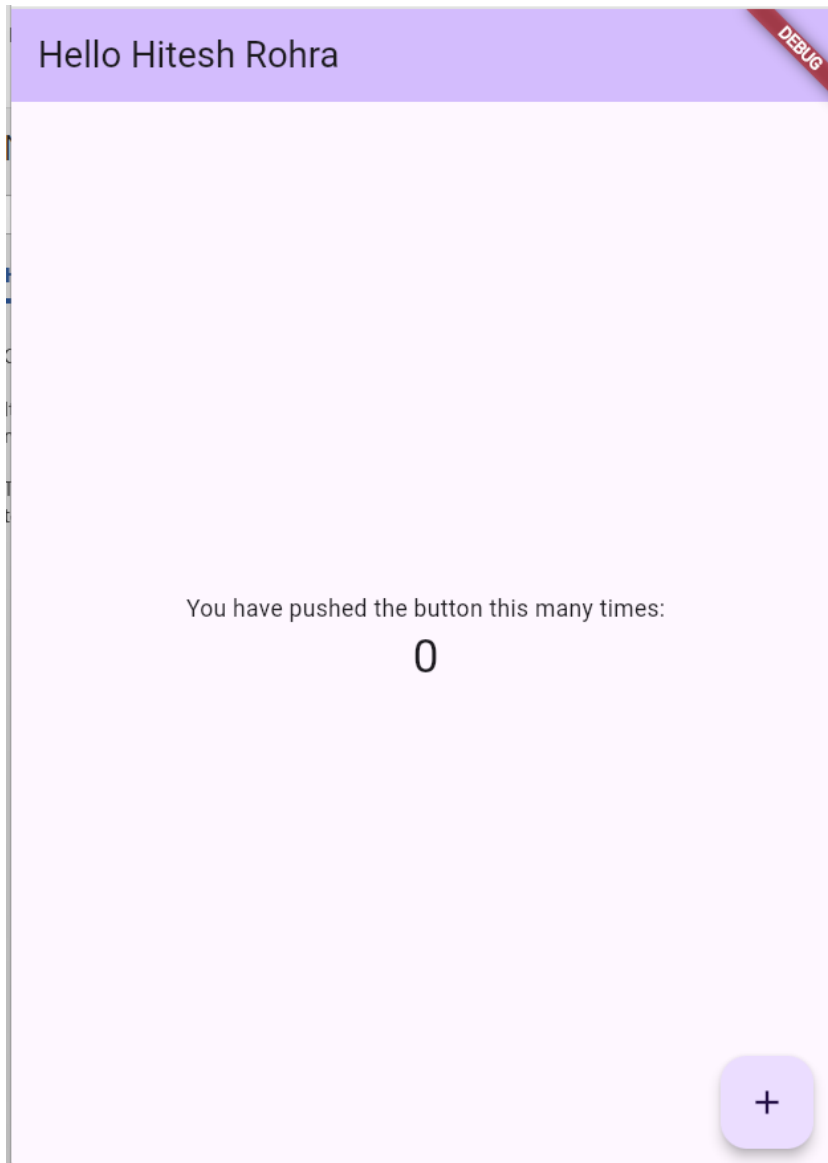
Step 11: - Now, install Flutter and Dart plugin for building Flutter application in Android Studio. These plugins provide a template to create a Flutter application, give an option to run and debug Flutter application in the Android Studio itself

Step 11.1: - Open the Android Studio and then go to File->Settings->Plugins. Now, search the Flutter plugin. If found, select Flutter plugin and click install



Step 11.2: - Restart the Android Studio

Step 12: - Go to File > New Project > Create Flutter Project, then select the project name and location, and click Next to proceed.



MAD & PWA Lab

Journal

Experiment No.	02
Experiment Title.	To design Flutter UI by including common widgets.
Roll No.	46
Name	Hitesh Rohra
Class	D15A
Subject	MAD & PWA Lab
Lab Outcome	LO2: Design and Develop interactive Flutter App by using widgets, layouts, gestures and animation
Grade:	

Experiment-2

Name: Hitesh Rohra

Class : D15A

Roll no: 46

Aim:

To design Flutter UI by including common widgets.

Theory:

Flutter follows a widget-based approach where everything in the UI is a widget. Widgets can be classified into two main types:

- **Stateless Widgets:** Do not change their state once built (e.g., Text, Container).
- **Stateful Widgets:** Can update dynamically based on user interaction (e.g., TextField, Checkbox).

Commonly Used Widgets in Flutter

(a) Scaffold Widget

The `Scaffold` widget provides the basic structure for a Flutter app, including an `AppBar`, `Drawer`, `FloatingActionButton`, and `BottomNavigationBar`. It is a fundamental widget used to create a standard screen layout in Flutter.

(b) Container Widget

A `Container` is a box model widget that can hold other widgets. It is commonly used for adding padding, margins, borders, and background decorations.

(c) Row and Column Widgets

- **Row:** Arranges widgets horizontally.
 - **Column:** Arranges widgets vertically.
- These two widgets are fundamental for designing layouts in Flutter.

(d) ListView Widget

The `ListView` widget is used for displaying a scrollable list of items. It is useful for showing large amounts of data dynamically.

(e) Stack Widget

The `Stack` widget is used to place widgets on top of each other. This is useful for creating overlapping UI elements such as banners, profile images, or layered designs.

(f) ElevatedButton Widget

The `ElevatedButton` widget is used for clickable buttons with a raised effect. It is a commonly used button in Flutter applications.

(g) TextField Widget

The `TextField` widget is used to take user input, such as entering a name, email, or password. It is commonly used in forms and authentication screens.

Home Page

This home page is designed using Flutter and consists of various widgets to create an

engaging UI for a movie app called **FilmyFun**.

Widgets Used:

1. **Scaffold Widget** – Provides the basic structure of the app.
2. **Container Widget** – Used for layout styling and decorations.
3. **Row & Column Widgets** – Organize UI elements horizontally and vertically.
4. **Text Widget** – Displays textual content.
5. **Image.asset Widget** – Loads images from local assets.
6. **CarouselSlider Widget** – Implements an image slider for featured movie posters.
7. **ListView Widget** – Displays a scrollable list of trending movies.
8. **Stack Widget** – Used for overlaying text and effects on movie images.

UI Features:

1. Header Section:

- Displays a welcome message: *"Hello, Manorath"*.
- Includes a wave emoji icon and a user profile picture.

2. App Title:

- The app name is displayed in a stylish way:
 - "Filmy" (White text)
 - "Fun" (Golden yellow text)

3. Carousel Image Slider:

- Showcases featured movie posters in a **carousel slider**.

- Users can swipe through movie images.

4. Trending Movies Section:

- Displays popular movies in a horizontal scrollable **ListView**.
- Each movie has:
 - A movie poster.
 - A title (e.g., *Infinity Wars*).
 - A genre (e.g., *Action, Adventure*).
 - A semi-transparent overlay for better readability.

Color Scheme:

- **Background:** Black for a cinematic feel.
- **Text Colors:** White and golden yellow for contrast.
- **Overlay Effects:** Black with transparency for better readability.

home.dart

Code:

```
import
'package:carousel_slider/carousel_slider.dart';

import 'package:flutter/material.dart';

class home extends StatefulWidget {

  const home({super.key});

  @override

  State<home> createState() =>
    _homeState();

}

class _homeState extends State<home> {
```

```

final List<String>imageUrls = [
    "images/infinity.jpg",
    "images/salman.jpg",
    "images/shahrukhmovies.png",
];

@override
Widget build(BuildContext context) {
    return Scaffold(
        backgroundColor: Colors.black,
        body: Container(
            margin: EdgeInsets.only(top: 40.0,left: 20.0),
            child: Column(
                crossAxisAlignment:
                CrossAxisAlignment.start,
                children: [
                    Row(children: [
                        Image.asset("images/wave.png", height: 40,
                        width: 40, fit: BoxFit.cover,),
                        SizedBox(width: 10.0,),
                        Text("Hello, Manorath",
                            style: TextStyle(
                                color: Colors.white,
                                fontSize: 22.0,
                                fontWeight: FontWeight.bold),
                            ),

```

```

Spacer(),
                Padding(
                    padding: const EdgeInsets.only(right: 20.0),
                    child: ClipRRect(
                        borderRadius: BorderRadius.circular(60),
                        child: Image.asset(
                            "images/Screenshot (55).png",
                            height: 60,
                            width: 60,
                            fit: BoxFit.cover,),
                    ),
                ),
                ],
            ),
            SizedBox(height: 10.0),
            Text("Welcome To",
                style: TextStyle(
                    color: Color.fromARGB(186, 255, 255, 255),
                    fontSize: 19.0,
                    fontWeight: FontWeight.bold),),
            Row(children: [
                Text("Filmy",
                    style: TextStyle(
                        color: Colors.white,

```

```

fontSize: 36.0,
fontWeight: FontWeight.bold),),
Text("Fun",
style: TextStyle(
color: Color(0xffedb41d),
fontSize: 36.0,
fontWeight: FontWeight.bold),)
],
),
SizedBox(height: 20.0,),
Center(child:
CarouselSlider(
items: imageUrls.map((url) {
return Builder(builder: (context) {
return Container(
width: MediaQuery.of(context).size.width, //
Fixed incorrect comma
child: ClipRRect(
borderRadius: BorderRadius.circular(10),
child: Image.asset(url, fit: BoxFit.cover),
),
);
});
}).toList(), options: CarouselOptions(
height: 250,

```

```

autoplay: false,
enlargeCenterPage: true,
aspectRatio: 16/9,
viewportFraction: 1.0
),// Fixed missing .toList()
),
SizedBox(height: 25.0,),
Text("Top Trending Movies",
style: TextStyle(
color: Colors.white,
fontSize: 24.0,
fontWeight: FontWeight.bold),),
SizedBox(height: 20.0,),
Container(
height: 250,
child: ListView(
scrollDirection: Axis.horizontal,
children: [
Container(
decoration: BoxDecoration(border:
Border.all(color:
Colors.white),borderRadius:
BorderRadius.circular(20)),
child: Stack(
children: [

```



```

),
Container(
padding: EdgeInsets.only(left: 10.0),
margin: EdgeInsets.only(top: 180),
height: 220,
width: 180,
decoration:
BoxDecoration(color:
Colors.black45,
borderRadius:
BorderRadius.only(
bottomRight: Radius.circular(20),
bottomLeft: Radius.circular(20))),
child: Column(
crossAxisAlignment:
CrossAxisAlignment.start,
children: [
Text("Pushpa 2", style: TextStyle(color:
Colors.white, fontSize: 25.0, fontWeight:
FontWeight.bold),),
Text("Action, Thriller,", style: TextStyle(color:
Color.fromARGB(173, 255, 255, 255),
fontSize: 16.0, fontWeight:
FontWeight.bold),),
],
),
),

```

```

],
),
),
SizedBox(width: 20.0),
Container(
decoration: BoxDecoration(border:
Border.all(color:
Colors.white),borderRadius:
BorderRadius.circular(20)),
child: Stack(
children: [
ClipRRect(
borderRadius: BorderRadius.circular(20),
child: Image.asset("images/salman.jpg",
height: 220,
width: 180,
fit: BoxFit.cover,
),
),
Container(
padding: EdgeInsets.only(left: 10.0),
margin: EdgeInsets.only(top: 180),
height: 220,
width: 180,
decoration:
BoxDecoration(color:

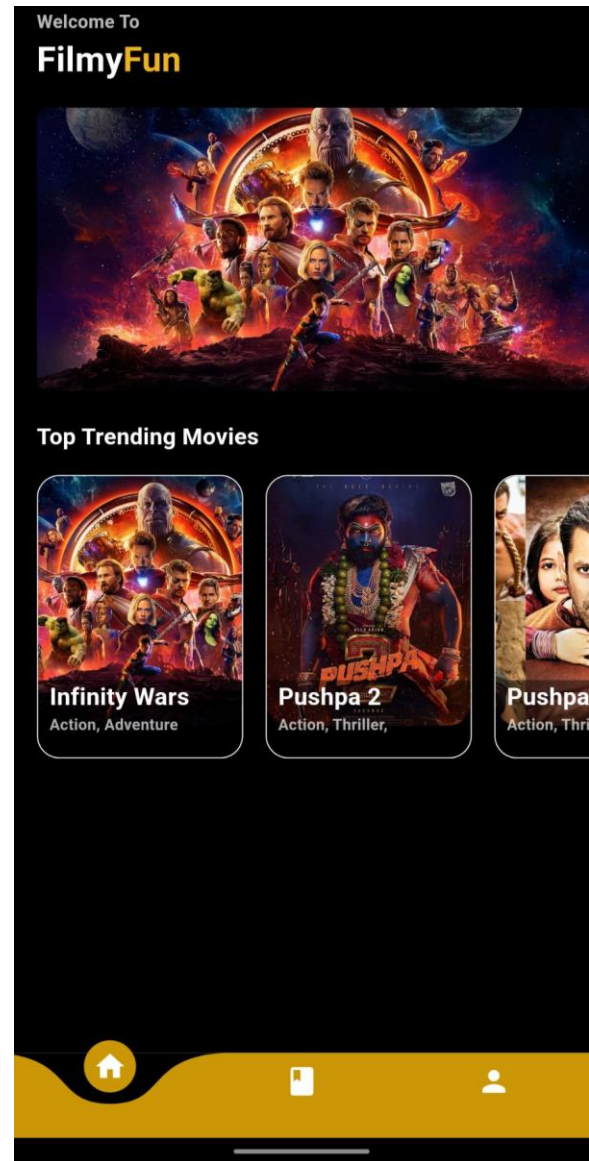
```



```

Colors.black45,
borderRadius:
BorderRadius.only(
bottomRight: Radius.circular(20),
bottomLeft: Radius.circular(20))),
child: Column(
crossAxisAlignment:
CrossAxisAlignment.start,
children: [
Text("Pushpa 2", style: TextStyle(color:
Colors.white, fontSize: 25.0, fontWeight:
FontWeight.bold),),
Text("Action, Thriller,", style: TextStyle(color:
Color.fromARGB(173, 255, 255, 255),
fontSize: 16.0, fontWeight:
FontWeight.bold),),
],
),
),
],
),
)
],
),
),
]),
));

```



MAD & PWA Lab Journal

Experiment No.	03
Experiment Title.	To include icons, images, fonts in Flutter app
Roll No.	46
Name	Hitesh Rohra
Class	D15A
Subject	MAD & PWA Lab
Lab Outcome	LO2: Design and Develop interactive Flutter App by using widgets, layouts, gestures and animation
Grade:	

EXPERIMENT NO: - 03

Hitesh Rohra

D15A/46

AIM: - To include icons, images, fonts in Flutter app.

Theory: -

Incorporating Icons, Images, and Custom Fonts into FilmyFun App

FilmyFun is a user-friendly movie ticket booking app that allows users to easily select showtimes and reserve tickets. A crucial aspect of creating an engaging and visually appealing app is incorporating various visual elements, such as icons, images, and custom fonts. These elements enhance the overall user experience and help the app stand out.

Like most modern mobile applications, FilmyFun uses resources such as assets and code to provide a dynamic user experience. Assets, such as images, icons, and custom fonts, are integral in delivering a rich and intuitive interface. These resources are bundled with the app and are made available during runtime.

Visual Elements and Their Role in the App

Visual elements—icons, images, and fonts—play an essential role in app development. These elements are used to improve usability, convey information quickly, and enhance branding.

1. **Enhanced User Experience:** Icons and images make the app visually attractive and easier to navigate.
2. **Information Conveyance:** Icons help reduce text and convey actions intuitively.
3. **Branding:** Custom icons and images align with FilmyFun's branding, making it memorable and recognizable.

Incorporating Icons in FilmyFun

Icons serve as intuitive visual shortcuts, providing a quick way for users to identify actions and navigate the app. Flutter provides a wide variety of built-in icons through the Icons class, and custom icons can be added via third-party packages like flutter_launcher_icons or font_awesome_flutter.

Example of adding an icon:

```
Icon(  
  Icons.movie,  
  size: 40,  
);
```

Adding Images in FilmyFun

Images are crucial for enhancing the visual experience in FilmyFun. Flutter allows for images to be added from multiple sources:

1. Assets (Stored Locally)

- To add an image stored locally in the project, place the image in the assets/images/ folder.
- Declare the image path in the pubspec.yaml file.

flutter:

assets:

- assets/images/movie_poster.png

- Display the image in the app using the Image.asset method:

```
Image.asset('assets/images/movie_poster.png');
```

Incorporating Custom Fonts in FilmyFun

While Flutter's default font is **Roboto**, you may want to add custom fonts to align with your app's branding and overall design aesthetic. Adding custom fonts is simple:

1. **Download the font** and place it in the assets/fonts/ folder of the project.
2. **Declare the font** in the pubspec.yaml file.

flutter:

fonts:

- family: CustomFont

fonts:

- asset: assets/fonts/CustomFont-Regular.ttf

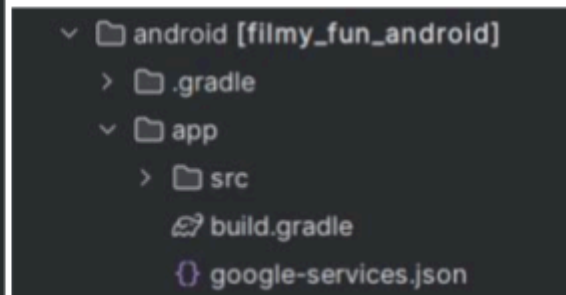
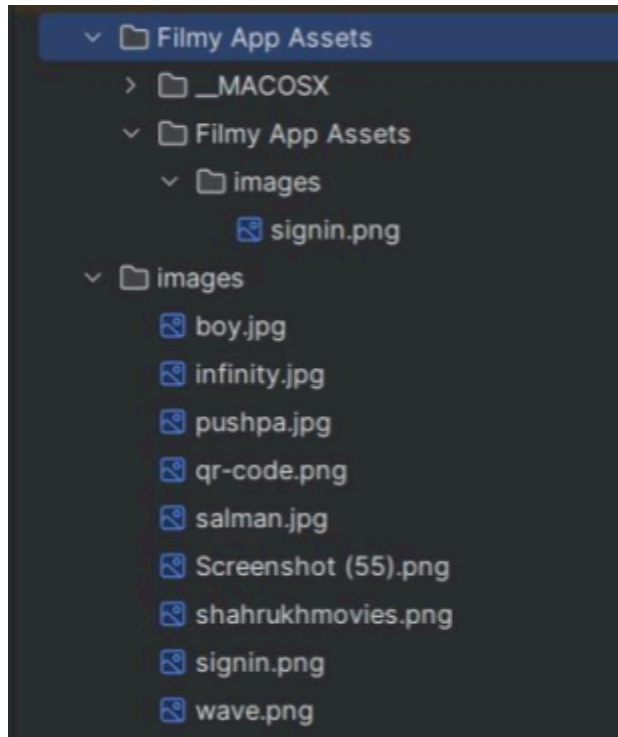
3. **Use the custom font** in your app:

Text(

'Welcome to FilmyFun!',

style: TextStyle(fontFamily: 'CustomFont', fontSize: 24),

);



Pubspec.yaml

name: filmy_fun

description: "FilmyFun-Movie Ticket Booking App"

The following line prevents the package from being accidentally published to

pub.dev using `flutter pub publish`. This is preferred for private packages.

publish_to: 'none' # Remove this line if you wish to publish to pub.dev

The following defines the version and build number for your application.

A version number is three numbers separated by dots, like 1.2.43

followed by an optional build number separated by a +.

Both the version and the builder number may be overridden in flutter

build by specifying --build-name and --build-number, respectively.

In Android, build-name is used as versionName while build-number used as versionCode.

Read more about Android versioning at <https://developer.android.com/studio/publish/versioning>

In iOS, build-name is used as CFBundleShortVersionString while build-number is used as

CFBundleVersion.

Read more about iOS versioning at

#

<https://developer.apple.com/library/archive/documentation/General/Reference/InfoPlistKeyReference/Articles/CoreFoundationKeys.html>

In Windows, build-name is used as the major, minor, and patch parts

of the product and file versions while build-number is used as the build suffix.

version: 1.0.0+1

environment:

sdk: ^3.6.2

Dependencies specify other packages that your package needs in order to work.

To automatically upgrade your package dependencies to the latest versions
consider running `flutter pub upgrade --major-versions`. Alternatively,
dependencies can be manually updated by changing the version numbers below to
the latest version available on pub.dev. To see which dependencies have newer
versions available, run `flutter pub outdated`.

dependencies:

```
flutter:  
  sdk: flutter  
carousel_slider: ^5.0.0  
curved_navigation_bar: ^1.0.3 # Use the latest version  
intl: ^0.18.0  
firebase_auth: ^5.4.2  
cloud_firestore: ^5.6.3  
random_string: ^2.3.1  
firebase_core: ^3.11.0  
shared_preferences:  
flutter_stripe:  
http:
```

The following adds the Cupertino Icons font to your application.

Use with the CupertinoIcons class for iOS style icons.

```
cupertino_icons: ^1.0.8
```

dev_dependencies:

```
flutter_test:  
  sdk: flutter
```

The "flutter_lints" package below contains a set of recommended lints
to # encourage good coding practices. The lint set provided by the package
is # activated in the `analysis\_options.yaml` file located at the root of your
package. See that file for information about deactivating specific
lint # rules and activating additional ones.

```
flutter_lints: ^5.0.0
```

For information on the generic Dart part of this file, see the
following page: <https://dart.dev/tools/pub/pubspec>

The following section is specific to Flutter packages.

flutter:

The following line ensures that the Material Icons font is
included with your application, so that you can use the icons in
the material Icons class.

```
uses-material-design: true
```

To add assets to your application, add an assets section, like this:

assets:

```
- images/  
# - images/a_dot_ham.jpeg
```

```
# An image asset can refer to one or more resolution-specific "variants", see  
# https://flutter.dev/to/resolution-aware-images
```

```
# For details regarding adding assets from package dependencies, see  
# https://flutter.dev/to/asset-from-package
```

```
# To add custom fonts to your application, add a fonts section here,  
# in this "flutter" section. Each entry in this list should have a  
# "family" key with the font family name, and a "fonts" key with a  
# list giving the asset and other descriptors for the font. For  
# example:
```

```
# fonts:
```

```
# - family: Schyler
```

```
#   fonts:
```

```
#     - asset:
```

```
fonts/Schyler-Regular.ttf #     - asset:
```

```
fonts/Schyler-Italic.ttf
```

```
#       style: italic
```

```
# - family: Trajan Pro
```

```
#   fonts:
```

```
#     - asset: fonts/TrajanPro.ttf
```

```
#     - asset:
```

```
fonts/TrajanPro_Bold.ttf #
```

```
    weight: 700
```

```
#
```

```
# For details regarding fonts from package dependencies,
```

```
# see https://flutter.dev/to/font-from-package
```

```
Home.dart
```

```
import 'package:carousel_slider/carousel_slider.dart';
```

```
import 'package:filmy_fun/pages/detail_page.dart';
```

```
import 'package:flutter/material.dart';
```

```
class home extends StatefulWidget {
```

```
  const home({super.key});
```

```
  @override
```

```
  State<home> createState() => _homeState();
```

```
}
```

```
class _homeState extends State<home> {
```

```
  final List<String> imageUrls = [
```

```
    "images/infinity.jpg",
```

```
    "images/salman.jpg",
```

```
    "images/shahrukhmovies.png",
```

```
  ];
```

```
  @override
```

```

Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: Colors.black,
    body: Container(
      margin: EdgeInsets.only(top: 40.0, left: 20.0),
      child:
        Column(crossAxisAlignment: CrossAxisAlignment.start, children: [
          Row(
            children: [
              Image.asset(
                "images/wave.png",
                height: 40,
                width: 40,
                fit: BoxFit.cover,
              ),
              SizedBox(
                width: 10.0,
              ),
              Text(
                "Hello, Manorath",
                style: TextStyle(
                  color: Colors.white,
                  fontSize: 22.0,
                  fontWeight: FontWeight.bold),
              ),
              Spacer(),
              Padding(
                padding: const EdgeInsets.only(right: 20.0),
                child: ClipRRect(
                  borderRadius: BorderRadius.circular(60),
                  child: Image.asset(
                    "images/Screenshot (55).png",
                    height: 60,
                    width: 60,
                    fit: BoxFit.cover,
                  ),
                ),
              ),
            ],
          ),
          SizedBox(height: 10.0),
          Text(
            "Welcome To",
            style: TextStyle(
              color: Color.fromARGB(186, 255, 255, 255),
              fontSize: 19.0,
              fontWeight: FontWeight.bold),
          ),
        ],
      ),
    ),
  );
}

```



```

Row(
  children: [
    Text(
      "Filmy",
      style: TextStyle(
        color: Colors.white,
        fontSize: 36.0,
        fontWeight: FontWeight.bold),
    ),
    Text(
      "Fun"
    ,
    style: TextStyle(
      color: Color(0xffedb41d),
      fontSize: 36.0,
      fontWeight: FontWeight.bold),
    )
  ],
),
 SizedBox(
  height: 20.0,
),
 Center(
  child: CarouselSlider(
    items: imageUrls.map((url) {
      return Builder(builder: (context) {
        return Container(
          width: MediaQuery.of(context)
            .size
            .width, // Fixed incorrect comma
          child: ClipRRect(
            borderRadius: BorderRadius.circular(10),
            child: Image.asset(url, fit: BoxFit.cover),
          ),
        );
      });
    }).toList(),
    options: CarouselOptions(
      height: 250,
      autoPlay: false,
      enlargeCenterPage: true,
      aspectRatio: 16 / 9,
      viewportFraction: 1.0), // Fixed missing .toList()
  )),
  SizedBox(
    height: 25.0,
  ),
  Text(
    "Top Trending Movies",

```

```

style: TextStyle(
  color: Colors.white,
  fontSize: 24.0,
  fontWeight: FontWeight.bold),
),
SizedBox(
  height: 20.0,
),
Container(
  height: 250,
  child: ListView(
    scrollDirection: Axis.horizontal,
    children: [
      GestureDetector(
        onTap: () {
          Navigator.push(
            context,
            MaterialPageRoute(
              builder: (context) =>
                DetailPage(
                  image:
                    "images/infinity.jpg", name:
                    "Infinity Wars",
                  shortdetail: "Action,
                    Adventure", moviedetail:
                    "Avengers: Infinity War is a 2018 American superhero film based on the
                    Marvel Comics superhero team the Avengers Produced by Marvel Studios and distributed by Walt
                    Disney Studios Motion Pictures, it is the sequel to The Avengers (2012) and Avengers: Age of Ultron
                    (2015), and the 19th film in the Marvel Cinematic Universe (MCU)",
                  price: '50',
                )),
        ),
      child: Container(
        decoration: BoxDecoration(
          border: Border.all(color: Colors.white),
          borderRadius: BorderRadius.circular(20)),
        child: Stack(
          children: [
            ClipRRect(
              borderRadius: BorderRadius.circular(20),
              child: Image.asset(
                "images/infinity.jpg",
                height: 220,
                width: 180,
                fit: BoxFit.cover,
              ),
            ),
          ],
        ),
        Container(
          padding: EdgeInsets.only(left: 10.0),
          margin: EdgeInsets.only(top: 180),

```

[illegible]


```

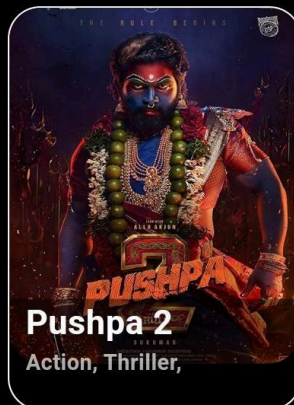
Container(
  padding: EdgeInsets.only(left: 10.0),
  margin: EdgeInsets.only(top: 180),
  height: 220,
  width: 180,
  decoration: BoxDecoration(
    color: Colors.black45,
    borderRadius: BorderRadius.only(
      bottomRight: Radius.circular(20),
      bottomLeft:
        Radius.circular(20))),
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Text(
        "Pushpa 2",
        style: TextStyle(
          color:
            Colors.white,
          fontSize: 25.0,
          fontWeight: FontWeight.bold),
      ),
      Text(
        "Action, Thriller",
        style: TextStyle(
          color: Color.fromARGB(173, 255, 255, 255),
          fontSize: 16.0,
          fontWeight: FontWeight.bold),
      ),
    ],
  ),
),
),
),
],
),
));
}
}

```

Welcome To
FilmyFun

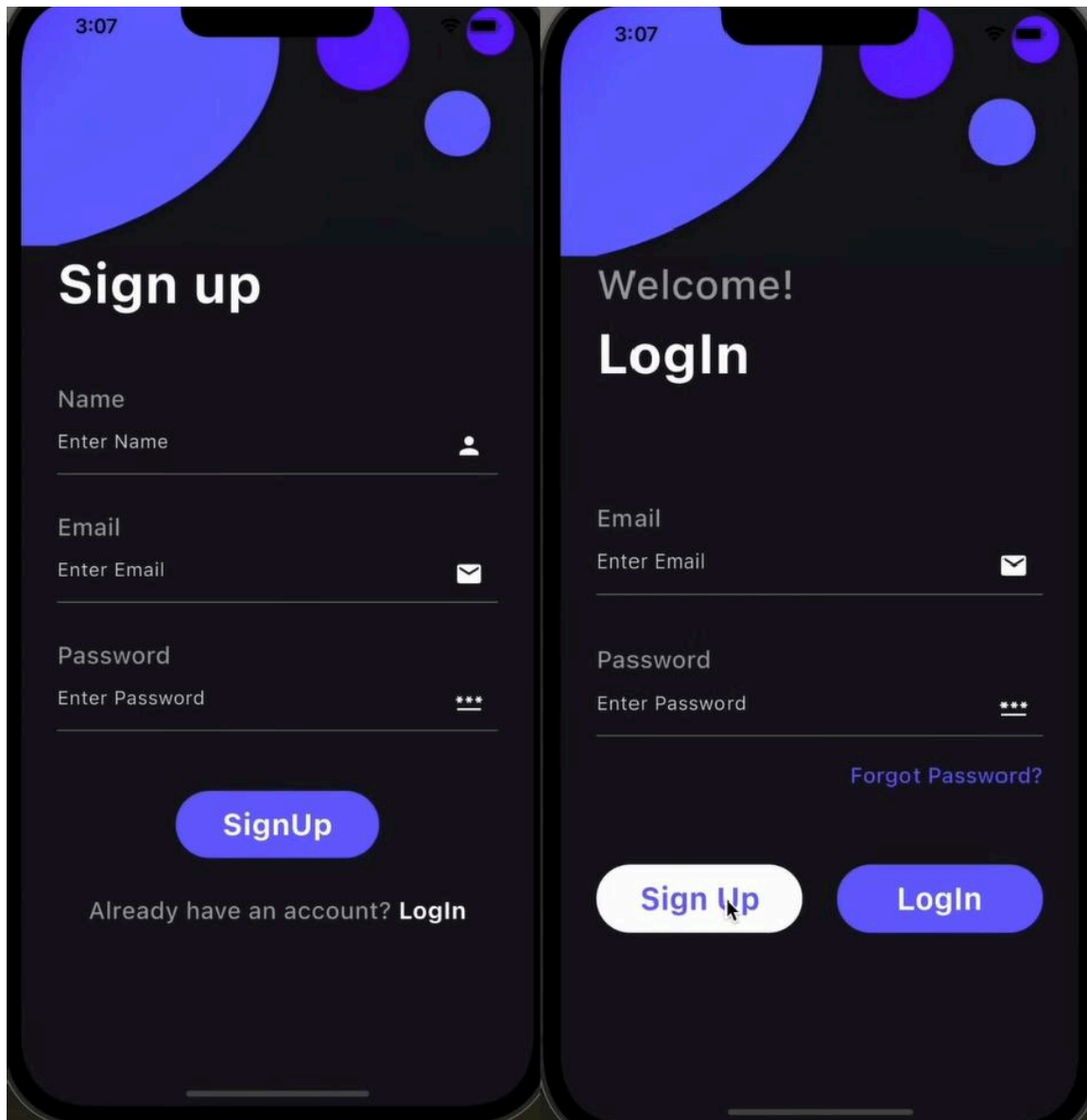


Top Trending Movies



ICONS:

```
buildInputField("Name", "Enter Name", Icons.person, namecontroller),  
buildInputField("Email", "Enter Email", Icons.email, mailcontroller),  
buildInputField("Password", "Enter Password", Icons.lock, passwordcontroller, obscureText: true),  
const SizedBox(height: 30.0),
```



Conclusion

By incorporating icons, images, and custom fonts into the FilmyFun app, you can create an intuitive, visually appealing, and engaging user experience. These visual elements contribute to the overall aesthetic, enhance usability, and ensure your app stands out in a competitive market. Whether displaying movie posters, providing easy navigation with icons, or using custom fonts for unique branding, these resources will elevate the look and feel of your app.

MAD & PWA Lab

Journal

Experiment No.	04
Experiment Title.	To create an interactive Form using form widget
Roll No.	46
Name	Hitesh Rohra
Class	D15A
Subject	MAD & PWA Lab
Lab Outcome	LO2: Design and Develop interactive Flutter App by using widgets, layouts, gestures and animation
Grade:	

Experiment 4

Name: Hitesh Rohra

Batch:D15A-46

Aim:To create interactive form

SignUp page

Code:

```
import 'package:flutter/material.dart';

import 'package:filmy_fun/pages/login.dart';

class Signup extends StatefulWidget {

  const Signup({super.key});

  @override

  State<Signup> createState() => _SignupState();

}

class _SignupState extends State<Signup> {

  @override

  Widget build(BuildContext context) {

    return Scaffold(

      backgroundColor: Colors.black,

      body: Padding(

        padding: const EdgeInsets.symmetric(horizontal: 20.0), // Adds padding to align left

        child: Column(

          crossAxisAlignment: CrossAxisAlignment.start, // Aligns content to left

          children: [

            Image.asset("images/signin.png"),

            const SizedBox(height: 20.0), // Adds spacing
```

```
const Text(  
  "Welcome!",  
  style: TextStyle(  
    color: Color.fromARGB(157, 255, 255, 255),  
    fontSize: 34.0,  
    fontWeight: FontWeight.w500,  
  ),  
),  
  
const Text(  
  "SignUp",  
  style: TextStyle(  
    color: Colors.white,  
    fontSize: 45.0,  
    fontWeight: FontWeight.bold,  
  ),  
),  
  
const SizedBox(height: 30.0), // Adjust spacing  
  
const Text(  
  "Name",  
  style: TextStyle(  
    color: Colors.white,  
    fontSize: 20.0,  
    fontWeight: FontWeight.w500,  
  ),  
),
```

```

),

TextField(

  decoration: InputDecoration(

    hintText: "Enter Name",

    hintStyle: TextStyle(

      color: Colors.white,

    ),

    suffixIcon: Icon(Icons.password,

      color: Colors.white,),

    ),

  ),

  const SizedBox(height: 50.0), // Adjust spacing

  const Text(

    "Email",

    style: TextStyle(

      color: Colors.white,

      fontSize: 20.0,

      fontWeight: FontWeight.w500,

    ),

    ),

  TextField(

    decoration: InputDecoration(

      hintText: "Enter Email",

      hintStyle: TextStyle(

```

```
color: Colors.white,

),

suffixIcon: Icon(Icons.email, color: Colors.white,),

),

const SizedBox(height: 50.0), // Adjust spacing

const Text(

  "Password",

  style: TextStyle(

    color: Colors.white,

    fontSize: 20.0,

    fontWeight: FontWeight.w500,

  ),

),

TextField(

  decoration: InputDecoration(

    hintText: "Enter Password",

    hintStyle: TextStyle(

      color: Colors.white,

    ),

    suffixIcon: Icon(Icons.password,

      color: Colors.white,),

  ),

),
```

```
const SizedBox(height: 50.0), // Adjust spacing
```

```
const Text(
```

```
"Confirm Password",
```

```
style: TextStyle(
```

```
color: Colors.white,
```

```
fontSize: 20.0,
```

```
fontWeight: FontWeight.w500,
```

```
),
```

```
),
```

```
TextField(
```

```
decoration: InputDecoration(
```

```
hintText: "Enter Password",
```

```
hintStyle: TextStyle(
```

```
color: Colors.white,
```

```
),
```

```
suffixIcon: Icon(Icons.password,
```

```
color: Colors.white,),
```

```
),
```

```
),
```

```
SizedBox(height: 50.0),
```

```
Center(
```

```
child: Column(
```

```
children: [
```

```
Container(
```

```
width: 170,

padding: EdgeInsets.all(10),

decoration: BoxDecoration(

color: Colors.white,

borderRadius: BorderRadius.circular(30),

),

child: Text(

"SignUp",

style: TextStyle(

color: Color(0xff6b63ff),

fontSize: 25.0,

fontWeight: FontWeight.bold,

),

textAlign: TextAlign.center,

),

),

 SizedBox(height: 20.0), // Space between button and text

 Row(

  mainAxisAlignment: MainAxisAlignment.center,

  children: [

    Text(

      "Already have an account?",

      style: TextStyle(

        color: Color.fromARGB(175, 255, 255, 255),
```

$$),),),[),[),),[),),);}}$$

12:30

Va LTE5G28%

Welcome!

SignUp

Name

Enter Name

Email

Enter Email

Password

Enter Password

Confirm Password

Enter Password

SignUp

Already have an account?Login

Login page

Code:

```
import 'package:filmy_fun/pages/bottomnav.dart';

import 'package:flutter/material.dart';

import 'package:filmy_fun/pages/signup.dart';

class Login extends StatefulWidget {

  const Login({super.key});

  @override

  State<Login> createState() => _LoginState();

}

class _LoginState extends State<Login> {

  @override

  Widget build(BuildContext context) {

    return Scaffold(

      backgroundColor: Colors.black,

      body: Padding(

        padding: const EdgeInsets.symmetric(horizontal: 20.0), // Adds padding to align left

        child: Column(

          crossAxisAlignment: CrossAxisAlignment.start, // Aligns content to left

          children: [

            Image.asset("images/signin.png"),

            const SizedBox(height: 20.0), // Adds spacing

            const Text(

              "Welcome!",

              style: TextStyle(
```

```
color: Color.fromARGB(157, 255, 255, 255),

fontSize: 34.0,

fontWeight: FontWeight.w500,

),

),

const Text(

  "Login",

  style: TextStyle(

    color: Colors.white,

    fontSize: 45.0,

    fontWeight: FontWeight.bold,

  ),

),

const SizedBox(height: 50.0), // Adjust spacing

const Text(

  "Email",

  style: TextStyle(

    color: Colors.white,

    fontSize: 20.0,

    fontWeight: FontWeight.w500,

  ),

),

TextField(

  decoration: InputDecoration(
```

```
hintText: "Enter Email",

hintStyle: TextStyle(

color: Colors.white,

),

suffixIcon: Icon(Icons.email, color: Colors.white,),

),

),

const SizedBox(height: 50.0), // Adjust spacing

const Text(

"Password",

style: TextStyle(

color: Colors.white,

fontSize: 20.0,

fontWeight: FontWeight.w500,

),

),

TextField(

decoration: InputDecoration(

hintText: "Enter Password",

hintStyle: TextStyle(

color: Colors.white,

),

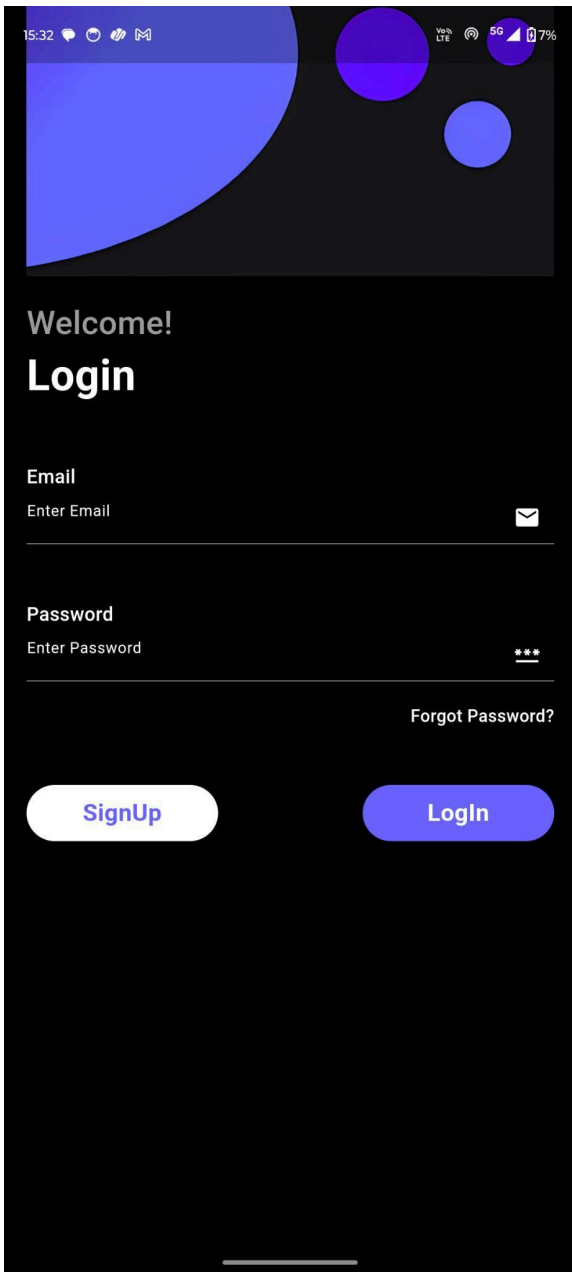
suffixIcon: Icon(Icons.password,

color: Colors.white,),
```

```
),  
  
),  
  
const SizedBox(height: 20.0),  
  
Row(  
  mainAxisAlignment: MainAxisAlignment.end,  
  children: [  
    Text(  
      "Forgot Password?",  
      style: TextStyle(  
        color: Colors.white,  
        fontSize: 18.0,  
        fontWeight: FontWeight.w500,  
      ),  
    )  
  ],),  
  SizedBox(height: 50.0,),  
  Row(  
    mainAxisAlignment: MainAxisAlignment.spaceBetween,  
    children: [  
      Row(  
        mainAxisAlignment: MainAxisAlignment.center,  
        children: [  
          GestureDetector(  
            onTap: () {Navigator.push(context, MaterialPageRoute(builder: (context)=> Signup()));},
```

```
child: Container(
  width: 170,
  padding: EdgeInsets.all(10),
  decoration: BoxDecoration(color: Colors.white,
    borderRadius: BorderRadius.circular(30)),
  child: Text("SignUp",style: TextStyle(color: Color(0xff6b63ff),
    fontSize: 25.0,
    fontWeight: FontWeight.bold),
    textAlign: TextAlign.center,
  ),
),
),
),
],
),
Row(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    GestureDetector(
      onTap: () {Navigator.push(context, MaterialPageRoute(builder: (context)=> Bottomnav()));},
      child: Container(
        width: 170,
        padding: EdgeInsets.all(10),
        decoration: BoxDecoration(
          color: Color(0xff6b63ff),
```

```
borderRadius: BorderRadius.circular(30)),  
  
child: Text("LogIn",style: TextStyle(color: Colors.white,  
  
fontSize: 25.0,  
  
fontWeight: FontWeight.bold),  
  
textAlign: TextAlign.center,  
  
),  
  
),  
  
),  
  
],  
  
)  
  
],  
  
)  
  
],  
  
,  
  
,  
  
);  
  
}  
  
}
```



MAD & PWA Lab

Journal

Experiment No.	05
Experiment Title.	To apply navigation, routing and gestures in Flutter App
Roll No.	46
Name	Hitesh Rohra
Class	D15A
Subject	MAD & PWA Lab
Lab Outcome	LO2: Design and Develop interactive Flutter App by using widgets, layouts, gestures and animation
Grade:	

EXPERIMENT NO: - 05

Hitesh Rohra

D15A/46

AIM: - To apply navigation, routing and gestures in Flutter App.

Theory: -

In Flutter, the screens and pages are known as routes, and these routes are just a widget. In Android, a route is similar to an Activity.

In any mobile app, navigating to different pages defines the workflow of the application, and the way to handle the navigation is known as routing. Flutter provides a basic routing class `MaterialPageRoute` and two methods `Navigator.push()` and `Navigator.pop()` that shows how to navigate between two routes. The following steps are required to start navigation in your application.

Gestures enable the app to respond to user interactions, making the application more dynamic and responsive.

➤ Navigation and Routing in Flutter

Navigation is the process of moving between different screens or pages in an app. Flutter provides a simple and effective way to handle this through the use of the `Navigator` widget and routes.

1. Using Navigator Widget

The `Navigator` widget manages a stack of routes, allowing for pushing and popping routes on the stack.

- Pushing a Route: To navigate to a new screen, use `Navigator.push()`.
- Popping a Route: To go back to the previous screen, use `Navigator.pop()`.

```
ElevatedButton(  
  onPressed: () {  
    Navigator.push(  
      context,  
      MaterialPageRoute(builder: (context) => SecondScreen()),  
    );  
  });  
);
```

2. Named Routes

Flutter also allows the use of named routes to navigate, which can make the routing process cleaner, especially in larger applications.

```
MaterialApp(  
  initialRoute: '/',  
  routes: {  
    '/': (context) => HomeScreen(),  
    '/second': (context) => SecondScreen(),  
  },  
);
```

Navigate to the route using `Navigator.pushNamed()`

```
Navigator.pushNamed(context, '/second');
```

Handling Gestures in Flutter

Gestures refer to user interactions with the app, such as taps, swipes, pinches, and drags. Flutter provides several widgets and gesture detectors to handle these interactions.

Tap Gestures

The most common gesture is the tap, which can be handled using the `GestureDetector` widget or specific buttons like `InkWell` or `ElevatedButton`.

Long Press Gesture

For long press gestures, Flutter provides the `onLongPress` callback in `GestureDetector` or `InkWell`.

Swipe and Drag Gestures

Flutter also provides swipe and drag gesture handling. The `onHorizontalDragUpdate` and `onVerticalDragUpdate` callbacks are used for dragging gestures.

Code:-

main.dart

```
import 'package:filmy_fun/pages/detail_page.dart';
import 'package:filmy_fun/service/constant.dart';
import 'package:firebase_core/firebase_core.dart';
import 'package:flutter/material.dart';
import 'package:filmy_fun/pages/login.dart';
import 'package:filmy_fun/pages/signup.dart';
import 'package:filmy_fun/pages/home.dart';
import 'package:filmy_fun/pages/bottomnav.dart';
import 'package:filmy_fun/pages/booking.dart';
import 'package:flutter_stripe/flutter_stripe.dart';

void main()async {
  WidgetsFlutterBinding.ensureInitialized();
  Stripe.publishableKey= publishedKey;
  await Firebase.initializeApp();
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  // This widget is the root of your application.
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'FilmyFun',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        // This is the theme of your application.
        //
        // TRY THIS: Try running your application with "flutter run". You'll see
        // the application has a purple toolbar. Then, without quitting the app,
        // try changing the seedColor in the colorScheme below to Colors.green
        // and then invoke "hot reload" (save your changes or press the "hot
        // reload" button in a Flutter-supported IDE, or press "r" if you used
        // the command line to start the app).
        //
        // Notice that the counter didn't reset back to zero; the application
        // state is not lost during the reload. To reset the state, use hot
        // restart instead.
        //
```

```

    // This works for code too, not just values: Most code changes can be
    // tested with just a hot reload.
    colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),
    useMaterial3: true,
  ),
  home: Signup(),
);
}
}

class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key, required this.title});

  // This widget is the home page of your application. It is stateful, meaning
  // that it has a State object (defined below) that contains fields that affect
  // how it looks.

  // This class is the configuration for the state. It holds the values (in this
  // case the title) provided by the parent (in this case the App widget) and
  // used by the build method of the State. Fields in a Widget subclass are
  // always marked "final".

  final String title;

  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  int _counter = 0;

  void _incrementCounter() {
    setState(() {
      // This call to setState tells the Flutter framework that something has
      // changed in this State, which causes it to rerun the build method below
      // so that the display can reflect the updated values. If we changed
      // _counter without calling setState(), then the build method would not be
      // called again, and so nothing would appear to happen.
      _counter++;
    });
  }

  @override
  Widget build(BuildContext context) {
    // This method is rerun every time setState is called, for instance as done
    // by the _incrementCounter method above.
    //
    // The Flutter framework has been optimized to make rerunning build methods

```

```

// fast, so that you can just rebuild anything that needs updating rather
// than having to individually change instances of widgets.
return Scaffold(
  appBar: AppBar(
    // TRY THIS: Try changing the color here to a specific color (to
    // Colors.amber, perhaps?) and trigger a hot reload to see the AppBar
    // change color while the other colors stay the same.
    backgroundColor: Theme.of(context).colorScheme.inversePrimary,
    // Here we take the value from the MyHomePage object that was created by
    // the App.build method, and use it to set our appBar title.
    title: Text(widget.title),
  ),
  body: Center(
    // Center is a layout widget. It takes a single child and positions it
    // in the middle of the parent.
    child: Column(
      // Column is also a layout widget. It takes a list of children and
      // arranges them vertically. By default, it sizes itself to fit its
      // children horizontally, and tries to be as tall as its parent.
      //
      // Column has various properties to control how it sizes itself and
      // how it positions its children. Here we use mainAxisAlignment to
      // center the children vertically; the main axis here is the vertical
      // axis because Columns are vertical (the cross axis would be
      // horizontal).
      //
      // TRY THIS: Invoke "debug painting" (choose the "Toggle Debug Paint"
      // action in the IDE, or press "p" in the console), to see the
      // wireframe for each widget.
      mainAxisAlignment: MainAxisAlignment.center,
      children: <Widget>[
        const Text(
          'You have pushed the button this many times:',
        ),
        Text(
          '$_counter',
          style: Theme.of(context).textTheme.headlineMedium,
        ),
      ],
    ),
  ),
  floatingActionButton: FloatingActionButton(
    onPressed: _incrementCounter,
    tooltip: 'Increment',
    child: const Icon(Icons.add),
  ), // This trailing comma makes auto-formatting nicer for build methods.
);

```

```
}  
}
```

Signup.dart

```
import 'package:filmy_fun/service/database.dart';  
import 'package:firebase_auth/firebase_auth.dart';  
import 'package:flutter/material.dart';  
import 'package:filmy_fun/pages/login.dart';  
import 'package:filmy_fun/service/shared_pref.dart';  
import 'package:random_string/random_string.dart';  
import 'bottomnav.dart';
```

```
class Signup extends StatefulWidget {  
  const Signup({super.key});
```

```
  @override  
  State<Signup> createState() => _SignupState();  
}
```

```
class _SignupState extends State<Signup> {  
  String email = "", password = "", name = "";  
  TextEditingController namecontroller = TextEditingController();  
  TextEditingController passwordcontroller = TextEditingController();  
  TextEditingController mailcontroller = TextEditingController();
```

```
  registration() async {  
    if (passwordcontroller.text.isNotEmpty &&  
        namecontroller.text.isNotEmpty &&  
        mailcontroller.text.isNotEmpty) {  
      try {  
        UserCredential userCredential = await FirebaseAuth.instance  
          .createUserWithEmailAndPassword(  
            email: mailcontroller.text, password: passwordcontroller.text);  
        String id = randomAlphaNumeric(10);  
        Map<String, dynamic> userInfoMap = {  
          "Name": namecontroller.text,  
          "Email": mailcontroller.text,  
          "Id": id,  
          "Image": ""  
        };  
        await SharedPreferencesHelper().saveUserDislayName(namecontroller.text);  
        await SharedPreferencesHelper().saveUserEmail(mailcontroller.text);  
        await SharedPreferencesHelper().saveUserID(id);  
        await SharedPreferencesHelper().saveUserImage("");  
        await DatabaseMethods().addUserDetails(userInfoMap, id);  
        ScaffoldMessenger.of(context).showSnackBar(SnackBar(  
          backgroundColor: Colors.green,  
          content: Text(  

```

```

        "Registered Successfully!",
        style: TextStyle(
          color: Colors.white,
          fontWeight: FontWeight.bold,
          fontSize: 20.0),
      ));
    Navigator.pushReplacement(
      context, MaterialPageRoute(builder: (context) => Bottomnav()));
  } on FirebaseAuthException catch (e) {
    if (e.code == 'weak-password') {
      ScaffoldMessenger.of(context).showSnackBar(SnackBar(
        backgroundColor: Colors.orangeAccent,
        content: Text(
          "Password Provided is too Weak",
          style: TextStyle(fontSize: 18.0),
        )),
    );
    } else if (e.code == "email-already-in-use") {
      ScaffoldMessenger.of(context).showSnackBar(SnackBar(
        backgroundColor: Colors.orangeAccent,
        content: Text(
          "Account Already exists",
          style: TextStyle(fontSize: 18.0),
        )),
    );
  }
}
}
}
}

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: Colors.black,
    body: SingleChildScrollView(
      child: Padding(
        padding: const EdgeInsets.symmetric(horizontal: 20.0),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const SizedBox(height: 50.0),
            Image.asset("images/signin.png"),
            const SizedBox(height: 20.0),
            const Text(
              "Welcome!",
              style: TextStyle(
                color: Color.fromARGB(157, 255, 255, 255),
                fontSize: 34.0,
                fontWeight: FontWeight.w500,
              ),
            ),
          ],
        ),
      ),
    ),
  );
}

```

```

    ),
    const Text(
      "SignUp",
      style: TextStyle(
        color: Colors.white,
        fontSize: 45.0,
        fontWeight: FontWeight.bold,
      ),
    ),
    const SizedBox(height: 30.0),
    buildInputField("Name", "Enter Name", Icons.person, namecontroller),
    buildInputField("Email", "Enter Email", Icons.email, mailcontroller),
    buildInputField("Password", "Enter Password", Icons.lock, passwordcontroller, obscureText:
true),
    const SizedBox(height: 30.0),
    Center(
      child: Column(
        children: [
          GestureDetector(
            onTap: registration,
            child: GestureDetector(
              onTap: () {
                if (namecontroller.text.isNotEmpty &&
                    mailcontroller.text.isNotEmpty &&
                    passwordcontroller.text.isNotEmpty) {
                  setState(() {
                    name = namecontroller.text;
                    email = mailcontroller.text;
                    password = passwordcontroller.text;
                  });
                  registration();
                }
              },
            child: Container(
              width: 170,
              padding: const EdgeInsets.all(12),
              decoration: BoxDecoration(
                color: Colors.white,
                borderRadius: BorderRadius.circular(30),
              ),
              child: const Text(
                "SignUp",
                style: TextStyle(
                  color: Color(0xff6b63ff),
                  fontSize: 25.0,
                  fontWeight: FontWeight.bold,
                ),
                textAlign: TextAlign.center,

```



```

        ),
        ),
        ),
    ),
    const SizedBox(height: 20.0),
    Row(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        const Text(
          "Already have an account?",
          style: TextStyle(
            color: Color.fromARGB(175, 255, 255, 255),
            fontSize: 18.0,
            fontWeight: FontWeight.w500,
          ),
        ),
        GestureDetector(
          onTap: () {
            Navigator.push(
              context,
              MaterialPageRoute(
                builder: (context) => const Login()),
            ),
          child: const Text(
            " LogIn",
            style: TextStyle(
              color: Colors.white,
              fontSize: 18.0,
              fontWeight: FontWeight.bold,
            ),
          ),
        ),
      ],
    ),
    const SizedBox(height: 30.0),
  ],
),
),
],
),
),
),
);
}

```

```

Widget buildInputField(String label, String hint, IconData icon,
  TextEditingController controller,
  {bool obscureText = false}) {

```

```

return Padding(
  padding: const EdgeInsets.only(bottom: 20.0),
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Text(
        label,
        style: const TextStyle(
          color: Colors.white,
          fontSize: 20.0,
          fontWeight: FontWeight.w500,
        ),
      ),
      TextField(
        controller: controller,
        obscureText: obscureText,
        cursorColor: Colors.white, // Cursor color set to white
        style: const TextStyle(color: Colors.white), // Text color set to white
        decoration: InputDecoration(
          hintText: hint,
          hintStyle: const TextStyle(color: Colors.white), // Hint text color set to white
          suffixIcon: Icon(icon, color: Colors.white),
          enabledBorder: const UnderlineInputBorder(
            borderSide: BorderSide(color: Colors.white), // Underline color
          ),
          focusedBorder: const UnderlineInputBorder(
            borderSide: BorderSide(color: Colors.white), // Underline when focused
          ),
        ),
      ),
    ],
  ),
);
}

```

Login.dart

```

import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:filmy_fun/pages/bottomnav.dart';
import 'package:filmy_fun/pages/home.dart';
import 'package:filmy_fun/service/database.dart';
import 'package:filmy_fun/service/shared_pref.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/material.dart';
import 'package:filmy_fun/pages/signup.dart';

```

```

class Login extends StatefulWidget {

```

```
const Login({super.key});
```

```
@override
```

```
State<Login> createState() => _LoginState();
```

```
}
```

```
class _LoginState extends State<Login> {
```

```
String email = "", password = "", myname = "", myid = "", myimage = "";
```

```
TextEditingController passwordcontroller = TextEditingController();
```

```
TextEditingController mailcontroller = TextEditingController();
```

```
userLogin() async {
```

```
  try {
```

```
    await FirebaseAuth.instance.signInWithEmailAndPassword(
```

```
      email: email,
```

```
      password: password,
```

```
    );
```

```
    QuerySnapshot querySnapshot =
```

```
    await DatabaseMethods().getUserByEmail(email);
```

```
    myname = "${querySnapshot.docs[0]["Name"]}";
```

```
    myid = "${querySnapshot.docs[0]["Id"]}";
```

```
    myimage = "${querySnapshot.docs[0]["Image"]}";
```

```
    await SharedPreferencesHelper().saveUserImage(myimage);
```

```
    await SharedPreferencesHelper().saveUserEmail(email);
```

```
    await SharedPreferencesHelper().saveUserDisplayName(myname);
```

```
    await SharedPreferencesHelper().saveUserID(myid);
```

```
    Navigator.push(context, MaterialPageRoute(builder: (context) => Bottomnav()));
```

```
  } on FirebaseAuthException catch (e) {
```

```
    String errorMessage = "";
```

```
    if (e.code == 'user-not-found') {
```

```
      ScaffoldMessenger.of(context).showSnackBar(SnackBar(
```

```
        content: Text("No user found for that email.",
```

```
        style: TextStyle(fontSize: 18.0, color: Colors.black),
```

```
      ));
```

```
    } else if (e.code == 'wrong-password') {
```

```
      ScaffoldMessenger.of(context).showSnackBar(SnackBar(
```

```
        backgroundColor: Colors.white,
```

```
        content: Text("Wrong password provided."
```

```
      ));
```

```
    }
```

```
  }
```

```
}
```

```
@override
```

```
Widget build(BuildContext context) {
```

```
  return Scaffold(
```

```
    backgroundColor: Colors.black,
```

```

body: Padding(
  padding: const EdgeInsets.symmetric(horizontal: 20.0),
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Image.asset("images/signin.png"),
      const SizedBox(height: 20.0),
      const Text(
        "Welcome!",
        style: TextStyle(
          color: Color.fromARGB(157, 255, 255, 255),
          fontSize: 34.0,
          fontWeight: FontWeight.w500,
        ),
      ),
      GestureDetector(
        onTap: () {
          if (mailcontroller.text.isNotEmpty &&
            passwordcontroller.text.isNotEmpty) {
            setState(() {
              email = mailcontroller.text;
              password = passwordcontroller.text;
              userLogin();
            });
          }
        },
        child: const Text(
          "Login",
          style: TextStyle(
            color: Colors.white,
            fontSize: 45.0,
            fontWeight: FontWeight.bold,
          ),
        ),
      ),
      const SizedBox(height: 50.0),
      const Text(
        "Email",
        style: TextStyle(
          color: Colors.white,
          fontSize: 20.0,
          fontWeight: FontWeight.w500,
        ),
      ),
      TextField(
        controller: mailcontroller,
        cursorColor: Colors.black,
        style: const TextStyle(color: Colors.white),
      ),
    ],
  ),
)

```

```

        decoration: const InputDecoration(
          hintText: "Enter Email",
          hintStyle: TextStyle(color: Colors.grey),
        ),
      ),
      const SizedBox(height: 50.0),
      const Text(
        "Password",
        style: TextStyle(
          color: Colors.white,
          fontSize: 20.0,
          fontWeight: FontWeight.w500,
        ),
      ),
      TextField(
        controller: passwordcontroller,
        cursorColor: Colors.black,
        style: const TextStyle(color: Colors.white),
        obscureText: true,
        decoration: const InputDecoration(
          hintText: "Enter Password",
          hintStyle: TextStyle(color: Colors.grey),
        ),
      ),
      const SizedBox(height: 20.0),
      Row(
        mainAxisAlignment: MainAxisAlignment.end,
        children: [
          Text(
            "Forgot Password?",
            style: TextStyle(
              color: Colors.white,
              fontSize: 18.0,
              fontWeight: FontWeight.w500,
            ),
          ),
        ],
      ),
      const SizedBox(height: 50.0),
      Row(
        mainAxisAlignment: MainAxisAlignment.spaceBetween,
        children: [
          GestureDetector(
            onTap: () {
              Navigator.push(context,
                MaterialPageRoute(builder: (context) => Signup()));
            },
            child: Container(

```

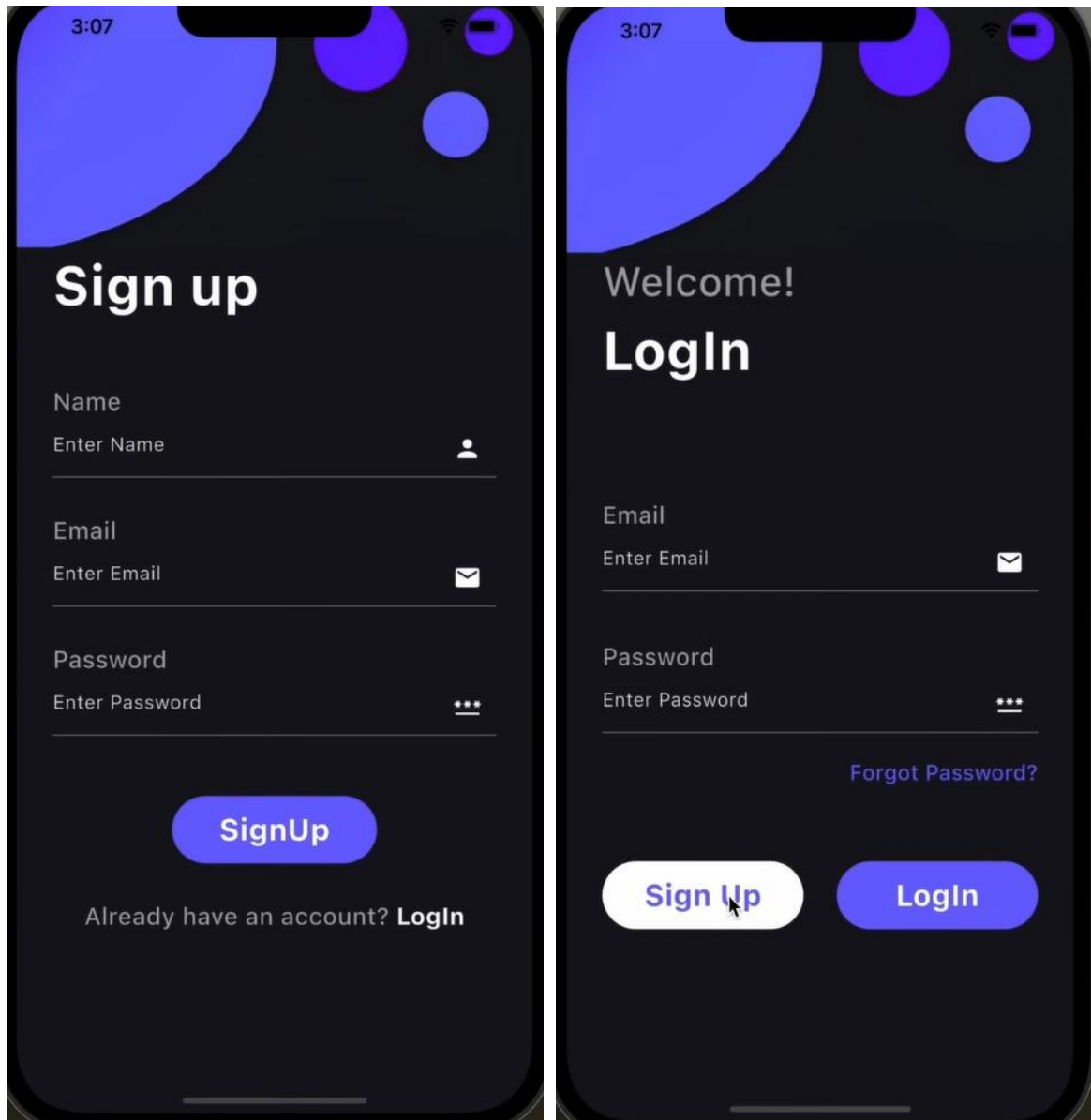
```

        width: 170,
        padding: EdgeInsets.all(10),
        decoration: BoxDecoration(
          color: Colors.white,
          borderRadius: BorderRadius.circular(30)),
        child: Text(
          "SignUp",
          style: TextStyle(
            color: Color(0xff6b63ff),
            fontSize: 25.0,
            fontWeight: FontWeight.bold),
          textAlign: TextAlign.center,
        ),
      ),
    ),
    GestureDetector(
      onTap: () {
        Navigator.push(
          context,
          MaterialPageRoute(
            builder: (context) => const Bottomnav()),
        ),
      ),
    child: Container(
      width: 170,
      padding: EdgeInsets.all(10),
      decoration: BoxDecoration(
        color: Color(0xff6b63ff),
        borderRadius: BorderRadius.circular(30)),
      child: Text(
        "LogIn",
        style: TextStyle(
          color: Colors.white,
          fontSize: 25.0,
          fontWeight: FontWeight.bold),
        textAlign: TextAlign.center,
      ),
    ),
  ),
],
),
],
),
),
);
}
}

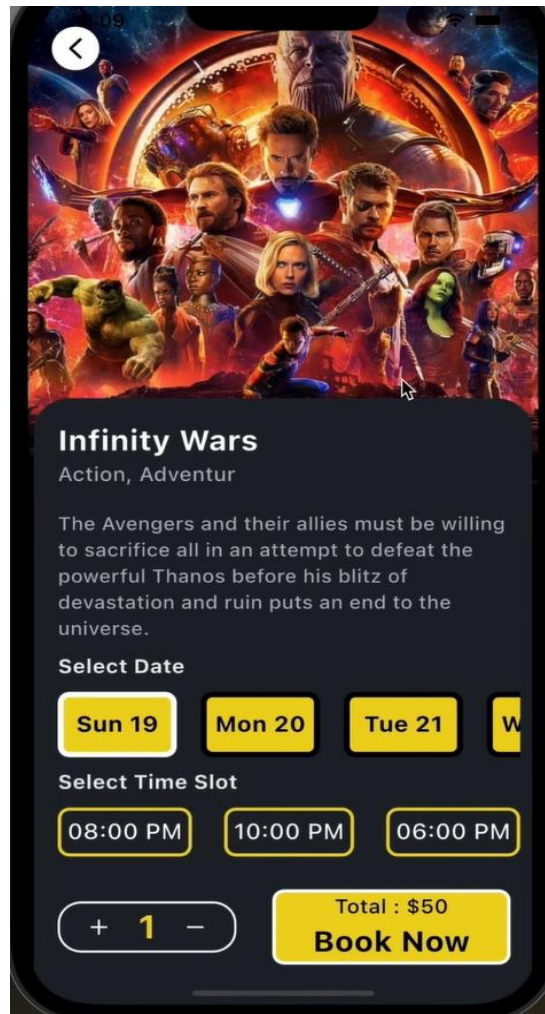
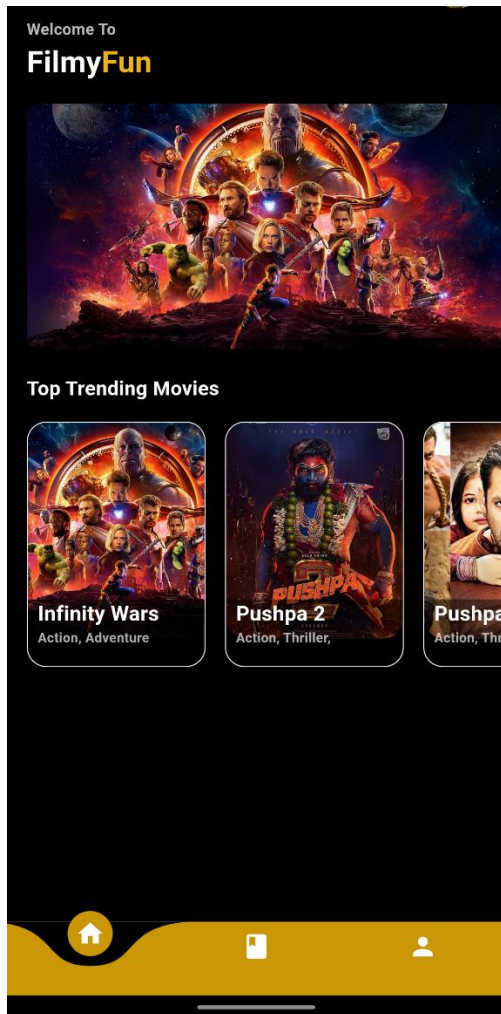
```

OUTPUT:

After clicking on Already have an account? It navigates to login



On Homepage after clicking on Movie image it navigates to Details page



MAD & PWA Lab

Journal

Experiment No.	06
Experiment Title.	To Connect Flutter UI with fireBase database
Roll No.	46
Name	Hitesh Rohra
Class	D15A
Subject	MAD & PWA Lab
Lab Outcome	LO3: Analyze and Build production ready Flutter App by incorporating backend services and deploying on Android / iOS
Grade:	

Experiment no - 6

Name – Hitesh Rohra

D15A – 46

AIM : To connect flutter UI with firebase database.

THEORY:

Introduction to Firebase and Flutter Integration

Firebase is a comprehensive platform developed by Google, designed to help developers build high-quality applications for both mobile and web. It provides essential services such as real-time databases, authentication, cloud storage, hosting, and much more. One of the most widely used Firebase services is the Firebase Realtime Database, which is a NoSQL cloud database that allows data to be stored and synced in real-time across all connected devices.

Flutter, on the other hand, is an open-source UI software development kit created by Google, which allows developers to build natively compiled applications for mobile, web, and desktop from a single codebase. Its rich set of pre-designed widgets and powerful tools makes Flutter an attractive option for developing visually appealing and performant applications. Integrating Firebase with Flutter allows developers to leverage the full potential of Firebase services in their applications. By using Firebase's Realtime Database, Flutter apps can achieve features such as real-time data synchronization, secure authentication, and cloud-based storage. This combination enables developers to create powerful, scalable, and feature-rich mobile and web applications.

Setting Up Firebase in Flutter :

To connect a Flutter app with Firebase, the following steps are typically followed:

1. **Creating a Firebase Project:** To start using Firebase with Flutter, the first step is to create a Firebase project in the Firebase Console. Once the project is created, developers can associate their Flutter app with the Firebase project by following the platform-specific instructions for Android or iOS. This usually involves configuring API keys, downloading configuration files, and adding them to the Flutter project.

2. **Integrating Firebase SDK in Flutter:** After the Firebase project is set up, developers need to integrate Firebase's SDK into the Flutter app. This involves adding the necessary dependencies to the Flutter project's pubspec.yaml file. For Firebase's Realtime Database, the package `firebase_database` is used.

Additionally, Firebase's core SDK (`firebase_core`) must also be included to initialize Firebase services.

3. **Initializing Firebase:** Before any Firebase functionality can be used, it is essential to initialize Firebase in the Flutter app. This is done by calling `Firebase.initializeApp()` in the main entry point of the app (usually in the `main.dart` file). Firebase needs to be initialized before interacting with any Firebase services, such as the Realtime Database, Cloud Firestore, or Authentication. Connecting Firebase to a Flutter app enables developers to create robust, scalable, and real-time applications with ease. Firebase's Realtime Database offers a powerful, cloud-based solution for managing data in realtime, while Firebase Authentication ensures secure access control. By integrating Firebase with Flutter, developers can take advantage of real-time data synchronization, offline support, and a wide range of other Firebase features, allowing them to build feature-rich apps that meet modern user expectations.

PROGRAM:

Signup.dart

```
import 'package:filmyapp/pages/bottomnav.dart';
import 'package:filmyapp/pages/login.dart';
import 'package:filmyapp/service/database.dart';
import 'package:filmyapp/service/shared_pref.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/material.dart';

class SignUp extends StatefulWidget {
  const SignUp({super.key});

  @override
  State<SignUp> createState() => _SignUpState();
}

class _SignUpState extends State<SignUp> {
  String email = "", password = "", name = "";
  TextEditingController namecontroller = TextEditingController();
  TextEditingController passwordcontroller = TextEditingController();
  TextEditingController mailcontroller = TextEditingController();

  registration() async {
    if (namecontroller.text != "" && mailcontroller.text != "") {
      try {
        UserCredential userCredential = await FirebaseAuth.instance
          .createUserWithEmailAndPassword(email: email, password: password);

        String uid = userCredential.user!.uid;

        Map<String, dynamic> userInfoMap = {
          "Name": namecontroller.text,
          "Email": mailcontroller.text,
          "Id": uid,
          "Image":
            "https://firebasestorage.googleapis.com/v0/b/barberapp-
            ebcc1.appspot.com/o/icon1.png?alt=media&token=0fad24a5-a01b-4d67-b4a0-676fbc75b34a",
        };

        await SharedPreferenceHelper().saveUserDisplayName(namecontroller.text);
        await SharedPreferenceHelper().saveUserEmail(mailcontroller.text);
        await SharedPreferenceHelper().saveUserId(uid); // Save UID here
        await SharedPreferenceHelper().saveUserImage(userInfoMap["Image"]);
        await DatabaseMethods().addUserDetails(userInfoMap, uid);

        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(
```

```

        backgroundColor: Colors.green,
        content: Text(
          "Registered Successfully!",
          style: TextStyle(
            color: Colors.white,
            fontWeight: FontWeight.bold,
            fontSize: 20.0,
          ),
        ),
      ),
    ),
  );
  Navigator.pushReplacement(
    context, MaterialPageRoute(builder: (context) => BottomNav()));
} on FirebaseAuthException catch (e) {
  String errorMessage = e.code == 'weak-password'
    ? "Password Provided is too Weak"
    : "Account Already exists";

  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(
      backgroundColor: Colors.orangeAccent,
      content: Text(
        errorMessage,
        style: const TextStyle(fontSize: 18.0),
      ),
    ),
  );
}
}
}

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: Color(0xff14141d),
    body: SingleChildScrollView(
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Image.asset("images/signin.png", height: 200, fit: BoxFit.cover),
          Padding(
            padding:
              const EdgeInsets.symmetric(horizontal: 30.0, vertical: 40.0),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                const Text(
                  "Sign Up",
                  style: TextStyle(
                    color: Colors.white,
                    fontSize: 32.0,

```

```

        fontWeight: FontWeight.bold,
      ),
    ),
    const SizedBox(height: 20.0),
    const Text("Name",
      style: TextStyle(color: Colors.white, fontSize: 18.0)),
    TextField(
      controller: namecontroller,
      decoration: const InputDecoration(
        hintText: "Enter Name",
        hintStyle: TextStyle(color: Colors.white54),
        suffixIcon: Icon(Icons.person, color: Colors.white),
      ),
      style: const TextStyle(color: Colors.white),
      cursorColor: Colors.white,
    ),
    const SizedBox(height: 20.0),
    const Text("Email",
      style: TextStyle(color: Colors.white, fontSize: 18.0)),
    TextField(
      controller: mailcontroller,
      decoration: const InputDecoration(
        hintText: "Enter Email",
        hintStyle: TextStyle(color: Colors.white54),
        suffixIcon: Icon(Icons.email, color: Colors.white),
      ),
      style: const TextStyle(color: Colors.white),
      cursorColor: Colors.white,
    ),
    const SizedBox(height: 20.0),
    const Text("Password",
      style: TextStyle(color: Colors.white, fontSize: 18.0)),
    TextField(
      controller: passwordcontroller,
      obscureText: true,
      decoration: const InputDecoration(
        hintText: "Enter Password",
        hintStyle: TextStyle(color: Colors.white54),
        suffixIcon: Icon(Icons.lock, color: Colors.white),
      ),
      style: const TextStyle(color: Colors.white),
      cursorColor: Colors.white,
    ),
    const SizedBox(height: 30.0),
    Center(
      child: GestureDetector(
        onTap: () {
          if (namecontroller.text != "" &&
            mailcontroller.text != "" &&
            passwordcontroller.text != "") {
            setState() {

```

```

        name = namecontroller.text;
        email = mailcontroller.text;
        password = passwordcontroller.text;
    });
    registration();
  }
},
child: Container(
  width: 160,
  padding: const EdgeInsets.all(12),
  decoration: BoxDecoration(
    color: Color(0xff6b63ff),
    borderRadius: BorderRadius.circular(30),
  ),
  child: const Center(
    child: Text(
      "Sign Up",
      style: TextStyle(
        color: Colors.white,
        fontSize: 20.0,
        fontWeight: FontWeight.bold,
      ),
    ),
  ),
),
),
),
),
),
const SizedBox(height: 20.0),
Center(
  child: GestureDetector(
    onTap: () => Navigator.push(
      context,
      MaterialPageRoute(builder: (context) => const LogIn()),
    ),
  child: const Text(
    "Already have an account? Log In",
    style:
      TextStyle(color: Color(0xff6b63ff), fontSize: 16.0),
  ),
),
),
),
),
),
),
),
);
}
}

```

Login.dart

```
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:filmyapp/pages/bottomnav.dart';
import 'package:filmyapp/pages/signup.dart';
import 'package:filmyapp/service/shared_pref.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:flutter/material.dart';

class LogIn extends StatefulWidget {
  const LogIn({super.key});

  @override
  State<LogIn> createState() => _LogInState();
}

class _LogInState extends State<LogIn> {
  String email = "", password = "";
  TextEditingController passwordcontroller = TextEditingController();
  TextEditingController mailcontroller = TextEditingController();

  userLogin() async {
    try {
      UserCredential userCredential = await FirebaseAuth.instance
        .signInWithEmailAndPassword(email: email, password: password);

      // Fetch user data from Firestore
      var userDoc = await FirebaseFirestore.instance
        .collection("users")
        .doc(userCredential.user!.uid)
        .get();

      if (userDoc.exists) {
        await SharedPreferencesHelper().saveUserId(userCredential.user!.uid);
        await SharedPreferencesHelper().saveUserDisplayName(userDoc['Name']);
        await SharedPreferencesHelper().saveUserEmail(userDoc['Email']);
        await SharedPreferencesHelper().saveUserImage(userDoc['Image']);
      } else {
        print("User document not found in Firestore.");
      }
    }

    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Text("Logged in successfully!", style: TextStyle(color: Colors.white)),
        backgroundColor: Colors.green,
        duration: Duration(seconds: 2),
      ),
    );

    Navigator.push(context, MaterialPageRoute(builder: (context) => BottomNav()));
  } on FirebaseAuthException catch (e) {
```

```
String errorMessage = e.code == 'user-not-found'  
  ? "No user found for that email."  
  : "Wrong password provided.";
```

```
ScaffoldMessenger.of(context).showSnackBar(  
  SnackBar(  
    backgroundColor: Colors.white,  
    content: Text(errorMessage, style: TextStyle(fontSize: 16.0, color: Colors.black)),  
  ),  
);  
}  
}
```

@override

```
Widget build(BuildContext context) {  
  return Scaffold(  
    backgroundColor: Color(0xff14141d),  
    body: SingleChildScrollView(  
      child: Column(  
        crossAxisAlignment: CrossAxisAlignment.start,  
        children: [  
          Image.asset("images/signin.png", height: 200, fit: BoxFit.cover),  
          Padding(  
            padding:  
              const EdgeInsets.symmetric(horizontal: 30.0, vertical: 30.0),  
            child: Column(  
              crossAxisAlignment: CrossAxisAlignment.start,  
              children: [  
                const Text(  
                  "Log In",  
                  style: TextStyle(  
                    color: Colors.white,  
                    fontSize: 32.0,  
                    fontWeight: FontWeight.bold,  
                  ),  
                ),  
                const SizedBox(height: 40.0),  
                const Text(  
                  "Email",  
                  style: TextStyle(color: Colors.white, fontSize: 18.0),  
                ),  
                TextField(  
                  controller: mailcontroller,  
                  decoration: const InputDecoration(  
                    hintText: "Enter Email",  
                    hintStyle: TextStyle(color: Colors.white54),  
                    suffixIcon: Icon(Icons.email, color: Colors.white),  
                  ),  
                  style: const TextStyle(color: Colors.white),  
                  cursorColor: Colors.white,  
                ),  
              ],  
            ),  
          ],  
        ),  
      ),  
    ),  
  );  
}
```

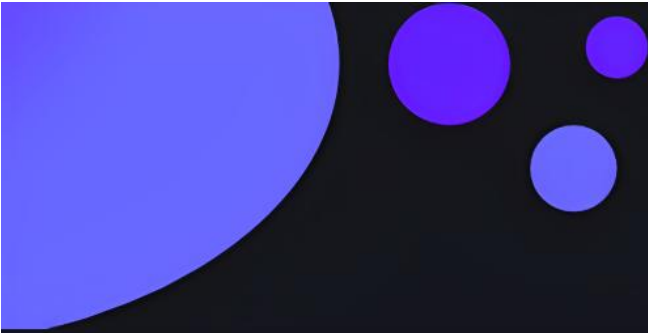


```

const SizedBox(height: 40.0),
const Text(
  "Password",
  style: TextStyle(color: Colors.white, fontSize: 18.0),
),
TextField(
  controller: passwordcontroller,
  obscureText: true,
  decoration: const InputDecoration(
    hintText: "Enter Password",
    hintStyle: TextStyle(color: Colors.white54),
    suffixIcon: Icon(Icons.lock, color: Colors.white),
  ),
  style: const TextStyle(color: Colors.white),
  cursorColor: Colors.white,
),
const SizedBox(height: 40.0),
Center(
  child: GestureDetector(
    onTap: () {
      if (mailcontroller.text != "" &&
        passwordcontroller.text != "") {
        setState(() {
          email = mailcontroller.text;
          password = passwordcontroller.text;
        });
      }
      userLogin();
    },
    child: Container(
      width: 160,
      padding: const EdgeInsets.all(12),
      decoration: BoxDecoration(
        color: Color(0xff6b63ff),
        borderRadius: BorderRadius.circular(30),
      ),
      child: const Center(
        child: Text(
          "Log In",
          style: TextStyle(
            color: Colors.white,
            fontSize: 20.0,
            fontWeight: FontWeight.bold,
          ),
        ),
      ),
    ),
  ),
),
const SizedBox(height: 20.0),
Center(

```

```
child: GestureDetector(
  onTap: () => Navigator.push(
    context,
    MaterialPageRoute(builder: (context) => const SignUp()),
  ),
child: const Text(
  "New user? Sign Up",
  style:
    TextStyle(color: Color(0xff6b63ff), fontSize: 16.0),
),
),
),
),
),
),
),
),
),
);
}
```



Log In

Email

Enter Email



Password

Enter Password



Log In

New user? [Sign Up](#)

Sign Up

Name

Enter Name



Email

Enter Email



Password

Enter Password



Sign Up

Already have an account? [Log In](#)

MAD & PWA Lab

Journal

Experiment No.	07
Experiment Title.	To write meta data of your Ecommerce PWA in a Web app manifest file to enable “add to homescreen feature”.
Roll No.	46
Name	Hitesh Rohra
Class	D15A
Subject	MAD & PWA Lab
Lab Outcome	LO4: Understand various PWA frameworks and their requirements
Grade:	

❖ AIM

To write meta data of your Food App PWA in a Web app manifest file to enable “add to homescreen feature”.

❖ Theory:-

Regular Web Application

A regular web application is a website designed to be accessible on various devices, ensuring that content adjusts dynamically to different screen sizes. Built using web technologies such as HTML, CSS, JavaScript, and Ruby, these applications function through a browser. While they can leverage some native device features, their functionality is dependent on browser compatibility. For instance, a feature may work on Google Chrome but not on Safari or Mozilla Firefox due to browser limitations.

Progressive Web Application (PWA)

A Progressive Web Application (PWA) is an advanced version of a regular web app, integrating additional features to enhance the user experience. PWAs combine the best elements of desktop and mobile applications, delivering a seamless experience across platforms.

❖ Key Differences Between PWAs and Regular Web Apps

1. Native-Like Experience

While both PWAs and regular web apps utilize standard web technologies like HTML, CSS, and JavaScript, PWAs offer a user experience similar to native mobile applications. PWAs can access device-specific features such as push notifications without relying on a particular browser, creating a smoother, more integrated experience.

2. Ease of Access

Unlike traditional mobile apps that require time-consuming downloads and storage space, PWAs can be installed directly via a URL. Users can add a PWA to their home screen with a simple link, eliminating installation complexities and ensuring easy access while keeping brand presence strong.

3. Enhanced Performance

PWAs utilize caching mechanisms to pre-load content, such as text, stylesheets, and images, allowing for faster loading times. This significantly improves user engagement and retention by reducing waiting periods, ultimately benefiting businesses by increasing interaction rates.

4. Improved User Engagement

PWAs efficiently leverage push notifications and native device features to keep users engaged. Unlike regular web apps, their functionality is not restricted by browser dependencies, enabling businesses to notify users about updates, offers, and promotions without disruptions.

5. Real-Time Updates

A major advantage of PWAs is their ability to update automatically without requiring users to download new versions from an app store. Developers can push updates directly from the server, ensuring that users always access the latest features and improvements instantly.

6. Search Engine Optimization (SEO) Benefits

Since PWAs function within web browsers, they can be indexed by search engines, improving their visibility in search results. This gives them a strategic advantage over native apps, which are limited to app store searches.

7. Cost-Effective Development

Unlike native mobile apps, PWAs do not require approval or submission to app stores, reducing development and maintenance costs.

❖ Advantages and Limitations of PWAs

➤ Advantages:

- **Progressive:** Compatible with all browsers and devices, following the principle of progressive enhancement.
- **Responsive:** Adapts to different screen sizes, including desktops, tablets, and smartphones.
- **App-Like Feel:** Mimics the experience of native applications in navigation and interaction.
- **Always Updated:** Service workers ensure real-time updates without requiring user intervention.
- **Secure:** Delivered over HTTPS, ensuring secure data transfer and protection against cyber threats.
- **SEO-Friendly:** Can be indexed by search engines, enhancing discoverability.
- **Re-Engagement:** Enables push notifications to encourage continued user interaction.
- **Installable:** Allows users to add the app to their home screen without app store downloads.
- **Offline Functionality:** Can function in low or no connectivity conditions using cached content.

➤ Limitations:

- **Higher Battery Consumption:** PWAs tend to consume more battery due to constant background processes.
- **Limited Hardware Access:** Some device features, such as advanced sensors and Bluetooth, may not be fully accessible.
- **Offline Mode Constraints:** Some offline capabilities remain limited, depending on browser support.
- **No App Store Presence:** PWAs cannot generate traffic from app store searches.
- **Lack of Centralized Control:** Unlike native apps, PWAs do not undergo an official approval process, potentially affecting credibility.

CODE:

Index.html :

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <link rel="icon" href="%PUBLIC_URL%/favicon.ico" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <meta name="theme-color" content="#000000" />
    <meta name="description" content="Web site created using create-react-app" />

    <link rel="manifest" href="%PUBLIC_URL%/manifest.json" />

    <link rel="apple-touch-icon" href="%PUBLIC_URL%/apple-touch-icon.png">
    <link rel="apple-touch-icon" sizes="192x192" href="%PUBLIC_URL%/logo.png">
    <link rel="apple-touch-icon" sizes="512x512" href="%PUBLIC_URL%/logo.png">

    <link rel="shortcut icon" href="%PUBLIC_URL%/favicon.ico" type="image/x-icon"
  />

    <link rel="stylesheet" href="https://use.fontawesome.com/releases/v5.15.4/css/all.css"
      integrity="sha384-
DyZ88mC6Up2uqS4h/KRgHuoGwBcD4Ng9SiP4dIRy0EXTlnuz47vAwmeGwVChigm
"
      crossorigin="anonymous" />

    <title>Delicia Food App</title>
  </head>
  <body>
    <noscript>You need to enable JavaScript to run this app.</noscript>
    <div id="root"></div>
  </body>
</html>
```

Manifest.json

```
"name": "Delice Food App",
"short_name": "Delice",
"start_url": "./index.html",
"scope": "./",
"icons": [
  {
    "src": "icon-192x192.png",
    "sizes": "192x192",
    "type": "image/png"
  },
  {
    "src": "icon-512x512.png",
    "sizes": "512x512",
    "type": "image/png"
  }
],
"theme_color": "#ffd31d",
"background_color": "#333",
"display": "standalone"
}
```

Icons

▼ PWA-PROJECT

▼ pwa-project

 icon-192x192.png

 icon-512x512.png

Google Dev

Application

▶  Manifest

 Service workers

 Storage

Storage

▶  Local storage

▶  Session storage

▶  Extension storage

 IndexedDB

▶  Cookies

 Private state tokens

 Interest groups

▶  Shared storage

 Cache storage

 Storage buckets

Background services

 Back/forward cache

 Background fetch

 Background sync

 Bounce tracking mitigati...

 Notifications

 Payment handler

 Periodic background sync

▶  Speculative loads

 Push messaging

 Reporting API

Frames

▶  top

Service workers

☐ Offline ☒ Update on reload ☐ Bypass for network

Service workers from other origins

[See all registrations](#)

❖ Output:-



App installed

Publisher: localhost:3000

Delicia Food App has been installed as an app on your device and will safely run in its own window. Launch it from the Start menu, Windows taskbar or your Desktop.

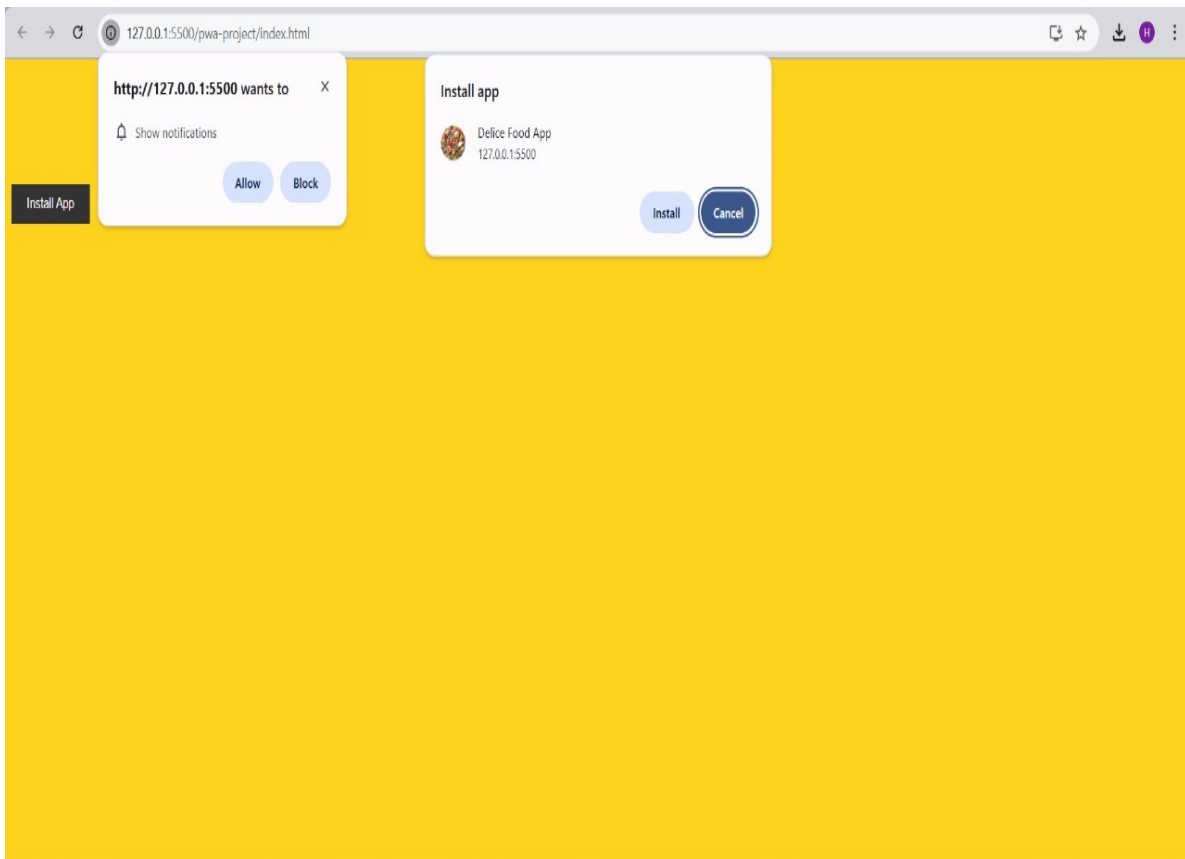
Allow this app to

- ☒ Pin to taskbar
- ☒ Pin to Start
- ☒ Create Desktop shortcut
- ☐ Auto-start on device login

Allow

Don't allow





MAD & PWA Lab

Journal

Experiment No.	08
Experiment Title.	To code and register a service worker, and complete the install and activation process for a new service worker for the E-commerce PWA
Roll No.	46
Name	Hitesh Rohra
Class	D15A
Subject	MAD & PWA Lab
Lab Outcome	LO5: Design and Develop a responsive User Interface by applying PWA Design techniques
Grade:	

MPL Experiment 8 (PWA)

Name: Devansh Wadhwani - 64
Kunal Punjabi - 43
Manav Punjabi - 44
Hitesh Rohra - 46

Class: D15A

Aim: To code and register a service worker, and complete the install and activation process for a new service worker for the Food App PWA.

Theory:

Service Worker

Service Worker is a script that works on browser background without user interaction independently. Also, It resembles a proxy that works on the user side. With this script, you can track network traffic of the page, manage push notifications and develop “offline first” web applications with Cache API.

Things to note about Service Worker:

- A service worker is a programmable network proxy that lets you control how network requests from your page are handled.
- Service workers only run over HTTPS. Because service workers can intercept network requests and modify responses, "man-in-the-middle" attacks could be very bad.
- The service worker becomes idle when not in use and restarts when it's next needed. You cannot rely on a global state persisting between events. If there is information that you need to persist and reuse across restarts, you can use IndexedDB databases.

What can we do with Service Workers?

- You can manage all network traffic of the page and do any manipulations. For example, when the page requests a CSS file, you can send plain text as a response or when the page requests an HTML file, you can send a png file as a response. You can also send a true response too.
- You can Cache
You can cache any request/response pair with Service Worker and Cache API and you can access these offline content anytime.
- You can manage Push Notifications
You can manage push notifications with Service Worker and show any information message to the user.
- You can Continue
Although Internet connection is broken, you can start any process with Background Sync of Service Worker.

What can't we do with Service Workers?

- You can't access the Window
You can't access the window, therefore, You can't manipulate DOM elements. But, you can communicate to the window through post Message and manage processes that you want.
- You can't work it on 80 Port
Service Worker just can work on HTTPS protocol. But you can work on localhost during development.

Codes:

//Serviceworker.js

```
const CACHE_NAME = 'delice-cache-v1';
const urlsToCache = [
  './',
  './index.html',
  './styles.css',
  './script.js',
  './icon-192x192.png',
  './icon-512x512.png'
];

// Install Service Worker
self.addEventListener('install', event => {
  event.waitUntil(
    caches.open(CACHE_NAME).then(cache => {
      console.log('Opened cache');
      return cache.addAll(urlsToCache);
    })
  );
});

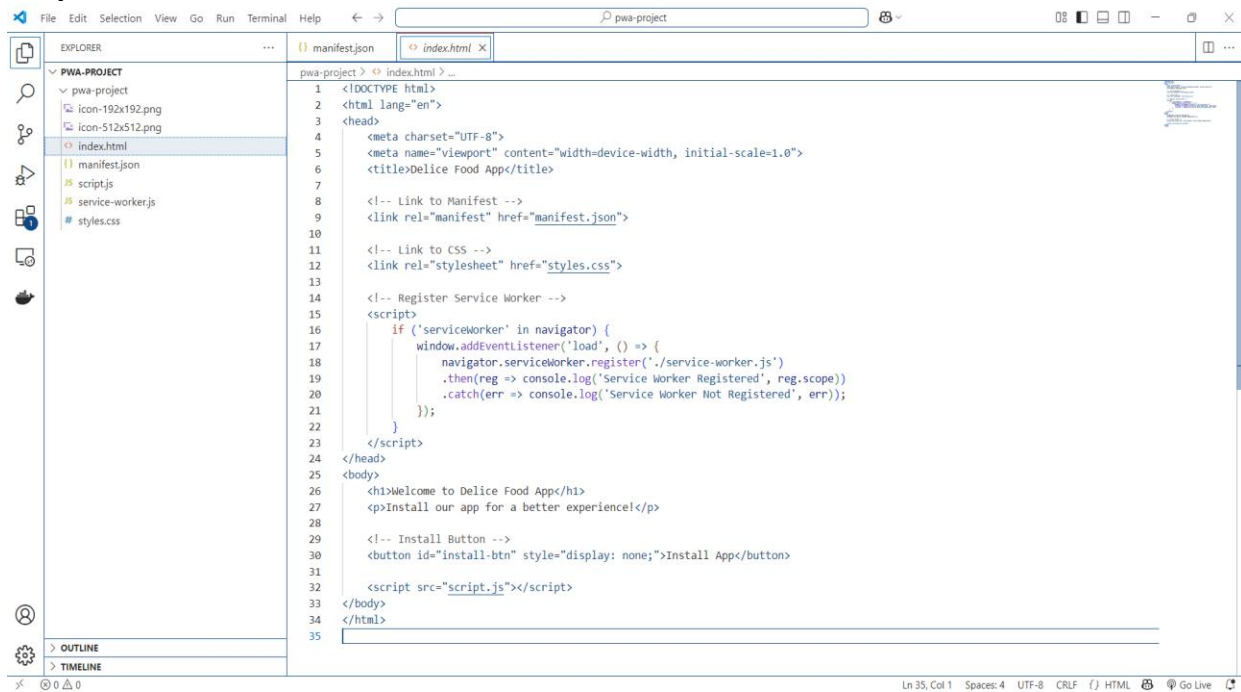
// Activate Service Worker (Old cache cleanup)
self.addEventListener('activate', event => {
  event.waitUntil(
    caches.keys().then(cacheNames => {
      return Promise.all(
        cacheNames.filter(cache => cache !==
CACHE_NAME).map(cache => caches.delete(cache))
      );
    })
  );
});

// Fetch Requests with Network Fallback
self.addEventListener('fetch', event => {
  event.respondWith(
    caches.match(event.request).then(response => {
      return response || fetch(event.request)
        .then(networkResponse => {
          return caches.open(CACHE_NAME).then(cache => {
            cache.put(event.request,
networkResponse.clone());
          });
        })
      );
    })
  );
});
```



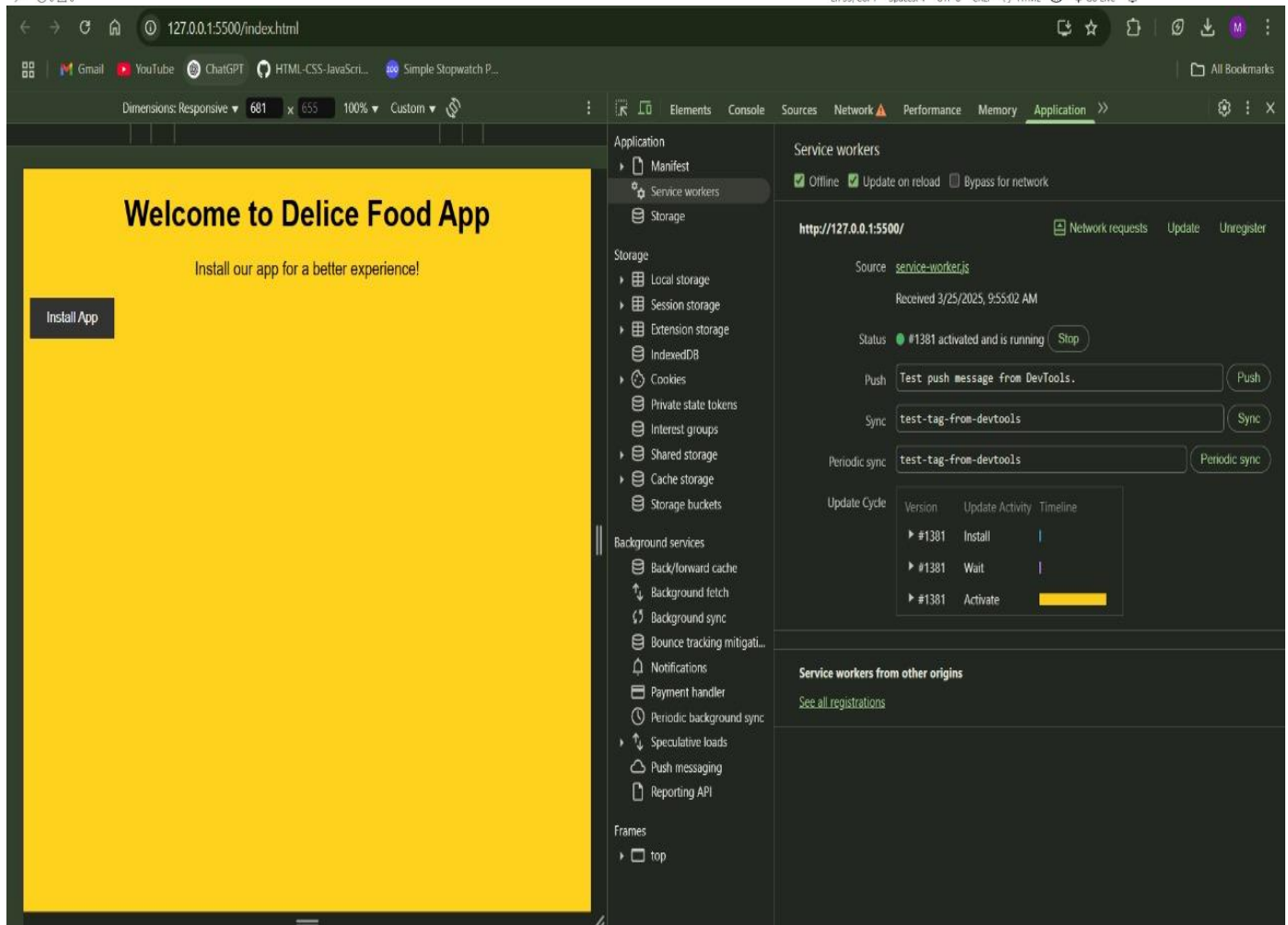
```
        return networkResponse;
    });
    });
}).catch(() => {
    return caches.match('./index.html'); // Offline fallback
})
);
});
```

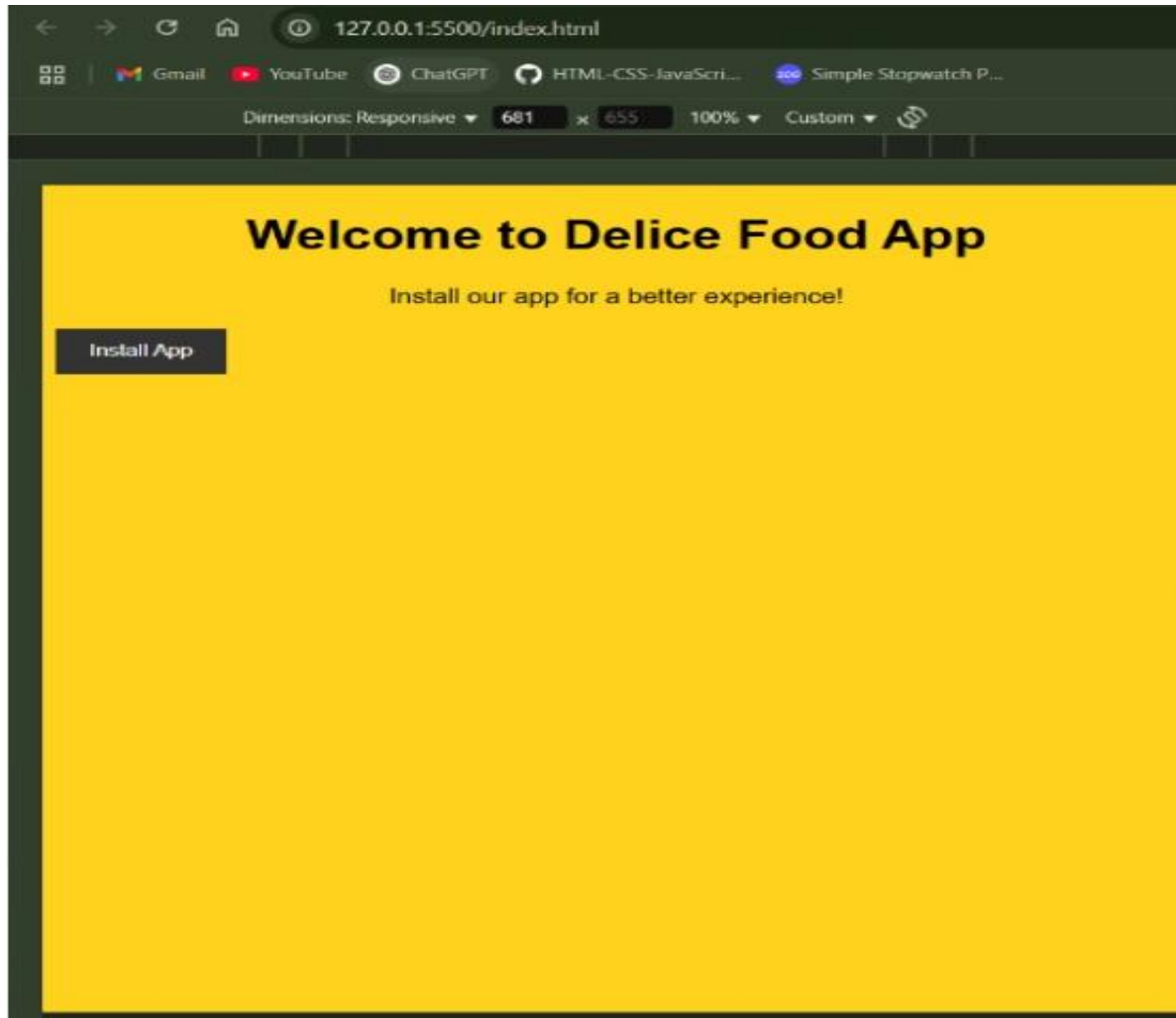
Output:



The screenshot shows the VS Code editor with a project named 'pwa-project'. The Explorer sidebar on the left lists files: 'icon-192x192.png', 'icon-512x512.png', 'index.html', 'manifest.json', 'script.js', 'service-worker.js', and 'styles.css'. The 'index.html' file is open in the editor, showing the following code:

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Delice Food App</title>
7
8   <!-- Link to Manifest -->
9   <link rel="manifest" href="manifest.json">
10
11   <!-- Link to CSS -->
12   <link rel="stylesheet" href="styles.css">
13
14   <!-- Register Service Worker -->
15   <script>
16     if ('serviceWorker' in navigator) {
17       window.addEventListener('load', () => {
18         navigator.serviceWorker.register('./service-worker.js')
19           .then(reg => console.log('Service Worker Registered', reg.scope))
20           .catch(err => console.log('Service Worker Not Registered', err));
21       });
22     }
23   </script>
24 </head>
25 <body>
26   <h1>Welcome to Delice Food App</h1>
27   <p>Install our app for a better experience!</p>
28
29   <!-- Install Button -->
30   <button id="install-btn" style="display: none;">Install App</button>
31
32   <script src="script.js"></script>
33 </body>
34 </html>
35
```





MAD & PWA Lab

Journal

Experiment No.	09
Experiment Title.	To implement Service worker events like fetch, sync and push for E-commerce PWA
Roll No.	46
Name	Hitesh Rohra
Class	D15A
Subject	MAD & PWA Lab
Lab Outcome	LO5: Design and Develop a responsive User Interface by applying PWA Design techniques
Grade:	

MPL Experiment 9 (PWA)

Name: Devansh Wadhwani - 64

Kunal Punjabi - 43

Manav Punjabi – 44

Hitesh Rohra - 46

Class: D15A

Aim: To implement Service worker events like fetch, sync and push for E-commerce PWA.

Theory:

Service Worker

Service Worker is a script that works on browser background without user interaction independently. Also, It resembles a proxy that works on the user side. With this script, you can track network traffic of the page, manage push notifications and develop “offline first” web applications with Cache API.

Things to note about Service Worker:

- A service worker is a programmable network proxy that lets you control how network requests from your page are handled.
- Service workers only run over HTTPS. Because service workers can intercept network requests and modify responses, "man-in-the-middle" attacks could be very bad.
- The service worker becomes idle when not in use and restarts when it's next needed. You cannot rely on a global state persisting between events. If there is information that you need to persist and reuse across restarts, you can use IndexedDB databases.
- Service workers make extensive use of promises, so if you're new to promises, then you should stop reading this and check out Promises, an introduction.

Fetch Event

You can track and manage page network traffic with this event. You can check existing cache, manage “cache first” and “network first” requests and return a response that you want.

Of course, you can use many different methods but you can find in the following example a “cache first” and “network first” approach. In this example, if the request’s and current location’s origin are the same (Static content is requested.), this is called “cacheFirst” but if you request a targeted external URL, this is called “networkFirst”.

- **CacheFirst** - In this function, if the received request has cached before, the cached response is returned to the page. But if not, a new response requested from the network.

- **NetworkFirst** - In this function, firstly we can try getting an updated response from the network, if this process completed successfully, the new response will be cached and returned. But if this process fails, we check whether the request has been cached before or not. If a cache exists, it is returned to the page, but if not, this is up to you. You can return dummy content or information messages to the page.

Sync Event

Background Sync is a Web API that is used to delay a process until the Internet connection is stable. We can adapt this definition to the real world; there is an e-mail client application that works on the browser and we want to send an email with this tool. Internet connection is broken while we are writing e-mail content and we didn't realize it. When completing the writing, we click the send button.

Push Event

This is the event that handles push notifications that are received from the server. You can apply any method with received data.

We can check in the following example.

"Notification.requestPermission();" is the necessary line to show notification to the user. If you don't want to show any notification, you don't need this line.

In the following code block is in sw.js file. You can handle push notifications with this event. In this example, I kept it simple. We send an object that has "method" and "message" properties. If the method value is "pushMessage", we open the information notification with the "message" property.

Code:

Serviceworker.js

```
const CACHE_NAME = 'delice-cache-v1';
const urlsToCache = [
  './',
  './index.html',
  './styles.css',
  './script.js',
  './icon-192x192.png',
  './icon-512x512.png'
];

// Install Service Worker
self.addEventListener('install', event => {
  event.waitUntil(
    caches.open(CACHE_NAME).then(cache => {
      console.log('Opened cache');
      return cache.addAll(urlsToCache);
    })
  );
});

// Activate Service Worker (Old cache cleanup)
self.addEventListener('activate', event => {
  event.waitUntil(
    caches.keys().then(cacheNames => {
      return Promise.all(
        cacheNames.filter(cache => cache !== CACHE_NAME).map(cache =>
          caches.delete(cache)
        )
      );
    })
  );
});

// Fetch Requests with Network Fallback
self.addEventListener('fetch', event => {
  event.respondWith(
    caches.match(event.request).then(response => {
      return response || fetch(event.request)
        .then(networkResponse => {
          return caches.open(CACHE_NAME).then(cache => {
            cache.put(event.request, networkResponse.clone());
            return networkResponse;
          });
        })
      );
    }).catch(() => {
      return caches.match('./index.html'); // Offline fallback
    })
  );
});
```

```
});
```

Push Notification Code:

1. Push Notification code:

```
// Push Notification Handling
```

```
self.addEventListener("push", (event) => {  
  console.log("Push notification received:", event);
```

```
  let data = { title: "New Notification", body: "Hello, this is a default message!" };
```

```
  if (event.data) {  
    try {  
      data = event.data.json();  
    } catch (error) {  
      console.warn("Push message is not JSON, using text instead.");  
      data.body = event.data.text();  
    }  
  }  
}
```

```
const options = {  
  body: data.body || "You have a new update!",  
  icon: "./assets/images/logo_tourly_192.png",  
  badge: "./assets/images/logo_tourly_192.png",  
  vibrate: [200, 100, 200],  
};
```

```
event.waitUntil(  
  self.registration.showNotification(data.title || "Hello Sannidhi", options)  
);  
});
```

```
// Click Event for Push Notifications
```

```
self.addEventListener("notificationclick", (event) => {  
  event.notification.close();  
  event.waitUntil(  
    clients.openWindow("http://localhost:5500/")  
  );  
});
```


Fetch Code :

```
self.addEventListener("fetch", (event) => {
  console.log("[Service Worker] Fetching:", event.request.url);

  if (event.request.method !== "GET") return; // Skip non-GET requests

  event.respondWith(
    caches.match(event.request).then((response) => {
      return response || fetch(event.request).then((networkResponse) => {
        return networkResponse;
      });
    }).catch(() => console.warn("[Service Worker] Network request failed:",
event.request.url))
  );
});

// Sync Event - Send Stored Data When Online
self.addEventListener("sync", (event) => {
  if (event.tag === SYNC_TAG) {
    console.log("[Service Worker] Sync event triggered:", SYNC_TAG);
    event.waitUntil(syncOfflineData());
  }
});

// Function to Sync Offline Data & Save to JSON
async function syncOfflineData() {
  const storedData = await getFromLocalStorage();
  if (!storedData.length) {
    console.log("[Service Worker] No offline data to sync.");
    return;
  }

  console.log("[Service Worker] Attempting to sync offline data...", storedData);

  try {
    const response = await fetch("http://localhost:3000/saveData", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(storedData)
    });

    if (response.ok) {
      console.log("[Service Worker] Successfully synced data:", storedData);
      await saveDataToJson(storedData); // Save data to JSON file
      clearLocalStorage(); // Clear after successful sync
    } else {
      console.warn("[Service Worker] Sync failed. Server response not OK.");
    }
  } catch (error) {
    console.error("[Service Worker] Sync failed:", error);
  }
}

// Function to Retrieve Data from Local Storage
function getFromLocalStorage() {
```

```

    return new Promise((resolve) => {
        const data = localStorage.getItem("offlineData");
        resolve(data ? JSON.parse(data) : []);
    });
}

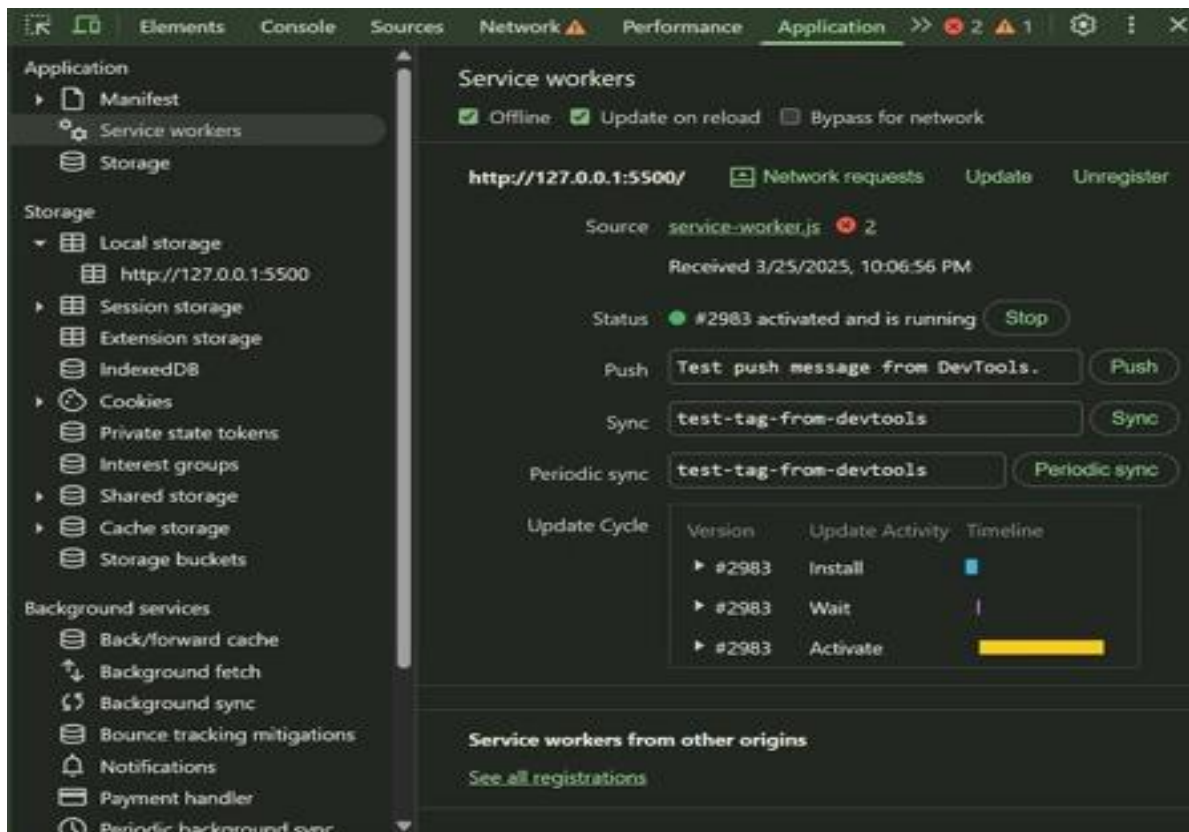
// Function to Clear Local Storage after Sync
function clearLocalStorage() {
    localStorage.removeItem("offlineData");
    console.log("[Service Worker] Local storage cleared after sync.");
}

// Save Synced Data to a JSON File
async function saveDataToJson(data) {
    try {
        const response = await fetch(JSON_FILE_URL, {
            method: "PUT", // Use PUT or POST based on server support
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify(data)
        });

        if (response.ok) {
            console.log("[Service Worker] Data saved to JSON file:", JSON_FILE_URL);
        } else {
            console.error("[Service Worker] Failed to save data to JSON.");
        }
    } catch (error) {
        console.error("[Service Worker] Error saving to JSON:", error);
    }
}

```

Output:



Sync Event

```
[Service Worker] Fetching: http://localhost:3000/saveData service-worker.js:113
✖ ▶ POST http://localhost:3000/saveData net::ERR_INTERNET_DISCONNECTED script.js:62
✖ ▶ Error: TypeError: Failed to fetch
    at HTMLFormElement.<anonymous> (script.js:62:32) script.js:73
Back Online: Attempting Cart Sync... (index):809
Cart Sync Registered (index):813
>
```

Push event

```
Push notification received: service-worker.js:93
  ▶ PushEvent {isTrusted: true, data: PushMessageData, type: 'push', target: ServiceWorkerGlob
    alScope, currentTarget: ServiceWorkerGlobalScope, ...} ⓘ
    isTrusted: true
    bubbles: false
    cancelBubble: false
    cancelable: false
    composed: false
    ▶ currentTarget: ServiceWorkerGlobalScope {clients: Clients, registration: ServiceWorkerReg
    ▶ data: PushMessageData {}
      defaultPrevented: false
      eventPhase: 0
      returnValue: true
    ▶ srcElement: ServiceWorkerGlobalScope {clients: Clients, registration: ServiceWorkerRegis
    ▶ target: ServiceWorkerGlobalScope {clients: Clients, registration: ServiceWorkerRegistrat
      timeStamp: 2785.800000011921
      type: "push"
    ▶ [[Prototype]]: PushEvent
  >
```

Welcome to Delice Food App

Install our app for a better experience!

Install App

MAD & PWA Lab

Journal

Experiment No.	10
Experiment Title.	To study and implement deployment of Ecommerce PWA to GitHub Pages.
Roll No.	46
Name	Hitesh Rohra
Class	D15A
Subject	MAD & PWA Lab
Lab Outcome	LO5: Design and Develop a responsive User Interface by applying PWA Design techniques
Grade:	

MPL Experiment 10 (PWA)

Class: D15A

Name:

Devansh Wadhwani – 64

Kunal Punjabi – 43

Manav Punjabi – 44

Hitesh Rohra - 46

Aim: To study and implement deployment of Food App PWA to GitHub Page.

Theory:

GitHub Pages

Public web pages are freely hosted and easily published. Public webpages hosted directly from your GitHub repository. Just edit, push, and your changes are live.

GitHub Pages provides the following key features:

- Blogging with Jekyll
- Custom URL
- Automatic Page Generator

Reasons for favoring this over Firebase:

- Free to use
- Right out of github
- Quick to set up

GitHub Pages is used by Lyft, CircleCI, and HubSpot.

GitHub Pages is listed in 775 company stacks and 4401 developer stacks.

Pros

- Very familiar interface if you are already using GitHub for your projects.
- Easy to set up. Just push your static website to the gh-pages branch and your website is ready.
- Supports Jekyll out of the box.
- Supports custom domains. Just add a file called CNAME to the root of your site, add an

A record in the site's DNS configuration, and you are done.

Cons

- The code of your website will be public, unless you pay for a private repository.
- Currently, there is no support for HTTPS for custom domains. It's probably coming soon though.
- Although Jekyll is supported, plug-in support is rather spotty.
- Firebase

The Realtime App Platform. Firebase is a cloud service designed to power real-time, collaborative applications. Simply add the Firebase library to your application to gain access to a shared data structure; any changes you make to that data are automatically synchronized with the Firebase cloud and with other clients within milliseconds.

Some of the features offered by Firebase are:

- Add the Firebase library to your app and get access to a shared data structure. Any changes made to that data are automatically synchronized with the Firebase cloud and with other clients within milliseconds.
- Firebase apps can be written entirely with client-side code, update in real-time out-of-the-box, interoperate well with existing services, scale automatically, and provide strong data security.
- Data Accessibility- Data is stored as JSON in Firebase. Every piece of data has its own URL which can be used in Firebase's client libraries and as a REST endpoint. These URLs can also be entered into a browser to view the data and watch it update in real-time.

Reasons for favoring over GitHub Pages:

- Realtime backend made easy
- Fast and responsive

Instacart, 9GAG, and Twitch are some of the popular companies that use Firebase. Firebase has a broader approval, being mentioned in 1215 company stacks & 4651 developers stacks.

Pros

- Hosted by Google. Enough said.
- Authentication, Cloud Messaging, and a whole lot of other handy services will be available to you.
- A real-time database will be available to you, which can store 1 GB of data.
- You'll also have access to a blob store, which can store another 1 GB of data.
- Support for HTTPS. A free certificate will be provisioned for your custom domain within 24 hours.

Cons

- Only 10 GB of data transfer is allowed per month. But this is not really a big problem, if you use a CDN or AMP.
- Command-line interface only.
- No in-built support for any static site generator.

Link to our GitHub repository:

<https://devanshwadhvani2004/Delicia-App-PWA/>

Link to our Hosted website:

<https://devanshwadhvani2004/Delicia-App-PWA/>

Github Screenshot:

The screenshot shows the GitHub interface for the repository 'Delicia-App-PWA' by user 'devanshwadhvani2004'. The repository is public and has 1 branch (main) and 0 tags. The file list shows the following files and their commit history:

File	Commit Message	Time
README.md	Initial commit	19 minutes ago
icon-192x192.png	Add files via upload	18 minutes ago
icon-512x512.png	Add files via upload	18 minutes ago
index.html	Add files via upload	18 minutes ago
manifest.json	Add files via upload	17 minutes ago
script.js	Add files via upload	17 minutes ago
service-worker.js	Add files via upload	17 minutes ago
styles.css	Add files via upload	17 minutes ago

Below the file list, there is a section for the README file, which is currently empty.

devanshwadhvani2004 / Delicia-App-PWA

Q Type to search

<> Code

🕒 Issues

🔀 Pull requests

⚙️ Actions

📁 Projects

📖 Wiki

🛡️ Security

📊 Insights

⚙️ Settings

⚙️ General

Access

👤 Collaborators

🔒 Moderation options

Code and automation

🌿 Branches

🏷️ Tags

📋 Rules

⚙️ Actions

🔗 Webhooks

🌐 Environments

💻 Codespaces

📄 Pages

Security

🔒 Advanced Security

🔑 Deploy keys

🔑 Secrets and variables

GitHub Pages

[GitHub Pages](#) is designed to host your personal, organization, or project pages from a GitHub repository.

Build and deployment

Source

Deploy from a branch

Branch

GitHub Pages is currently disabled. Select a source below to enable GitHub Pages for this repository. [Learn more about configuring the publishing source for your site.](#)

None Save

Visibility GitHub Enterprise

With a GitHub Enterprise account, you can restrict access to your GitHub Pages site by publishing it privately. You can use privately published sites to share your internal documentation or knowledge base with members of your enterprise. You can try GitHub Enterprise risk-free for 30 days. [Learn more about the visibility of your GitHub Pages site.](#)

Start free for 30 days

MAD & PWA Lab

Journal

Experiment No.	11
Experiment Title.	To use google Lighthouse PWA Analysis Tool to test the PWA functioning.
Roll No.	46
Name	Hitesh Rohra
Class	D15A
Subject	MAD & PWA Lab
Lab Outcome	LO6: Develop and Analyze PWA Features and deploy it over app hosting solution
Grade:	

MPL Experiment 11 (PWA)

Name: Devansh Wadhwani – 64

Kunal Punjabi – 43

Manav Punjabi – 44

Hitesh Rohra - 46

Class: D15A

Aim: To use google Lighthouse PWA Analysis Tool to test the PWA functioning.

Theory:

Google Lighthouse :

Google Lighthouse is a tool that lets you audit your web application based on a number of parameters including (but not limited to) performance, based on a number of metrics, mobile compatibility, Progressive Web App (PWA) implementations, etc. All you have to do is run it on a page or pass it a URL, sit back for a couple of minutes and get a very elaborate report, not much short of one that a professional auditor would have compiled in about a week.

The best part is that you have to set up almost nothing to get started. Let's begin by looking at some of the top features and audit criteria used by Lighthouse.

Key Features and Audit Metrics

Google Lighthouse has the option of running the Audit for Desktop as well as mobile version of your page(s). The top metrics that will be measured in the Audit are:

- **Performance:**

This score is an aggregation of how the page fared in aspects such as (but not limited to) loading speed, time taken for loading for basic frame(s), displaying meaningful content to the user, etc. To a layman, this score is indicative of how decently the site performs, with a score of 100 meaning that you figure in the 98th percentile, 50 meaning that you figure in the 75th percentile and so on.

- **PWA Score (Mobile):**

Thanks to the rise of Service Workers, app manifests, etc., a lot of modern web applications are moving towards the PWA paradigm, where the objective is to make the application behave as close as possible to native mobile

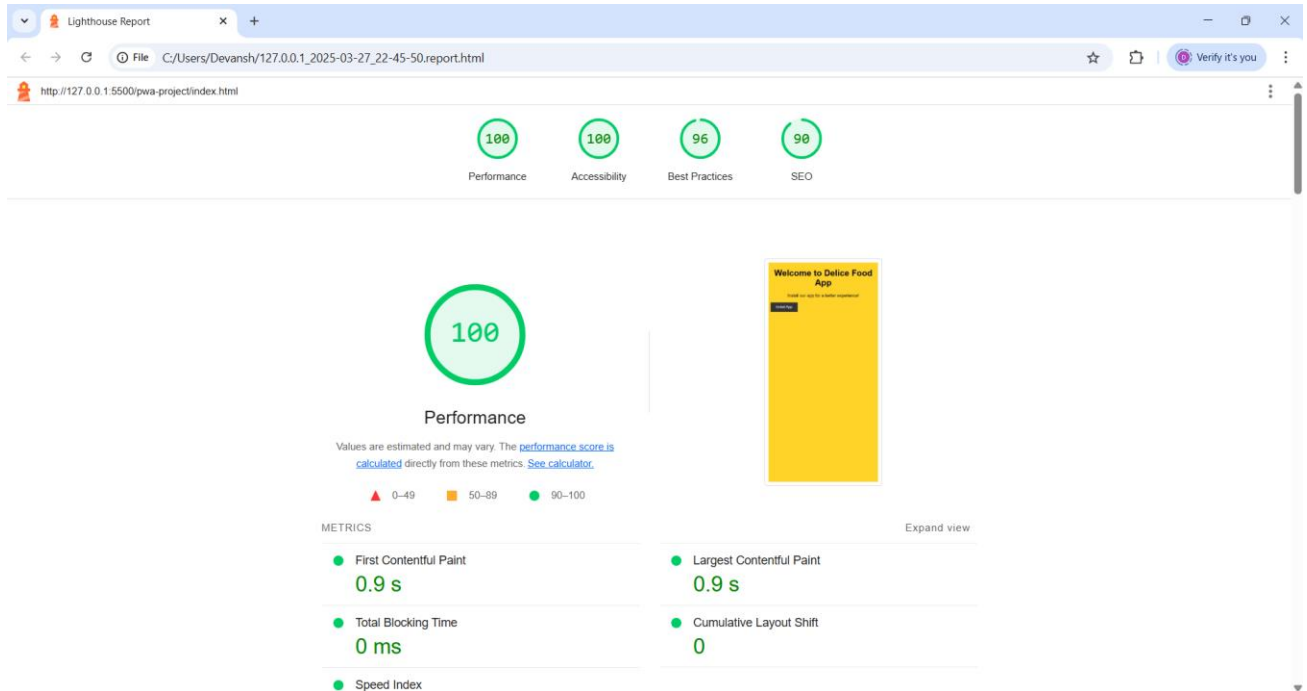
applications. Scoring points are based on the Baseline PWA checklist laid down by Google which includes Service Worker implementation(s), viewport handling, offline functionality, performance in script-disabled environments, etc.

- **Accessibility:**

As you might have guessed, this metric is a measure of how accessible your website is, across a plethora of accessibility features that can be implemented in your page (such as the 'aria-' attributes like aria-required, audio captions, button names, etc.). Unlike the other metrics though, Accessibility metrics score on a pass/fail basis i.e. if all possible elements of the page are not screen-reader friendly (HTML5 introduced features that would make pages easy to interpret for screen readers used by visually challenged people like tag names, tags such as <section>, <article>, etc.), you get a 0 on that score. The aggregate of these scores is your Accessibility metric score.

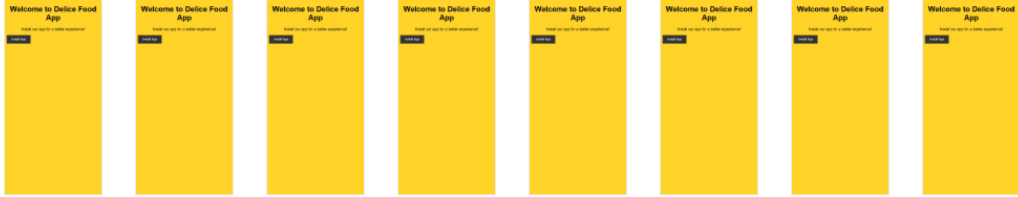
Best Practices: As any developer would know, there are a number of practices that have been deemed 'best' based on empirical data. This metric is an aggregation of many such points, including but not limited to: Use of HTTPS

Output:



0.9 s

 [View Treemap](#)



Show audits relevant to: [All](#) [FCP](#) [LCP](#)

DIAGNOSTICS

- ▲ Eliminate render-blocking resources — Potential savings of 290 ms ▼
- ▲ Page prevented back/forward cache restoration — 1 failure reason ▼
- Avoid chaining critical requests — 2 chains found ▼
- Largest Contentful Paint element — 910 ms ▼

More information about the performance of your application. These numbers don't [directly affect](#) the Performance score.

100

100

96

90

100

Accessibility

These checks highlight opportunities to [improve the accessibility of your web app](#). Automatic detection can only detect a subset of issues and does not guarantee the accessibility of your web app, so [manual testing](#) is also encouraged.

ADDITIONAL ITEMS TO MANUALLY CHECK (10)

Hide

- | | |
|---|---|
| ☐ Interactive controls are keyboard focusable | ▼ |
| ☐ Interactive elements indicate their purpose and state | ▼ |
| ☐ The page has a logical tab order | ▼ |
| ☐ Visual order on the page follows DOM order | ▼ |
| ☐ User focus is not accidentally trapped in a region | ▼ |
| ☐ The user's focus is directed to new content added to the page | ▼ |

100

100

96

90



Best Practices

GENERAL

- ▲ Browser errors were logged to the console ▼

TRUST AND SAFETY

- Ensure CSP is effective against XSS attacks ▼
- Use a strong HSTS policy ▼
- Ensure proper origin isolation with COOP ▼
- Mitigate clickjacking with XFO or CSP ▼



SEO

These checks ensure that your page is following basic search engine optimization advice. There are many additional factors Lighthouse does not score here that may affect your search ranking, including performance on [Core Web Vitals](#). [Learn more about Google Search Essentials](#).

CONTENT BEST PRACTICES

▲ Document does not have a meta description



Format your HTML in a way that enables crawlers to better understand your app's content.

ADDITIONAL ITEMS TO MANUALLY CHECK (1)

Show

Run these additional validators on your site to check additional SEO best practices.

Conclusion:

Thus we successfully used google Lighthouse PWA Analysis Tool for testing the PWA functioning.